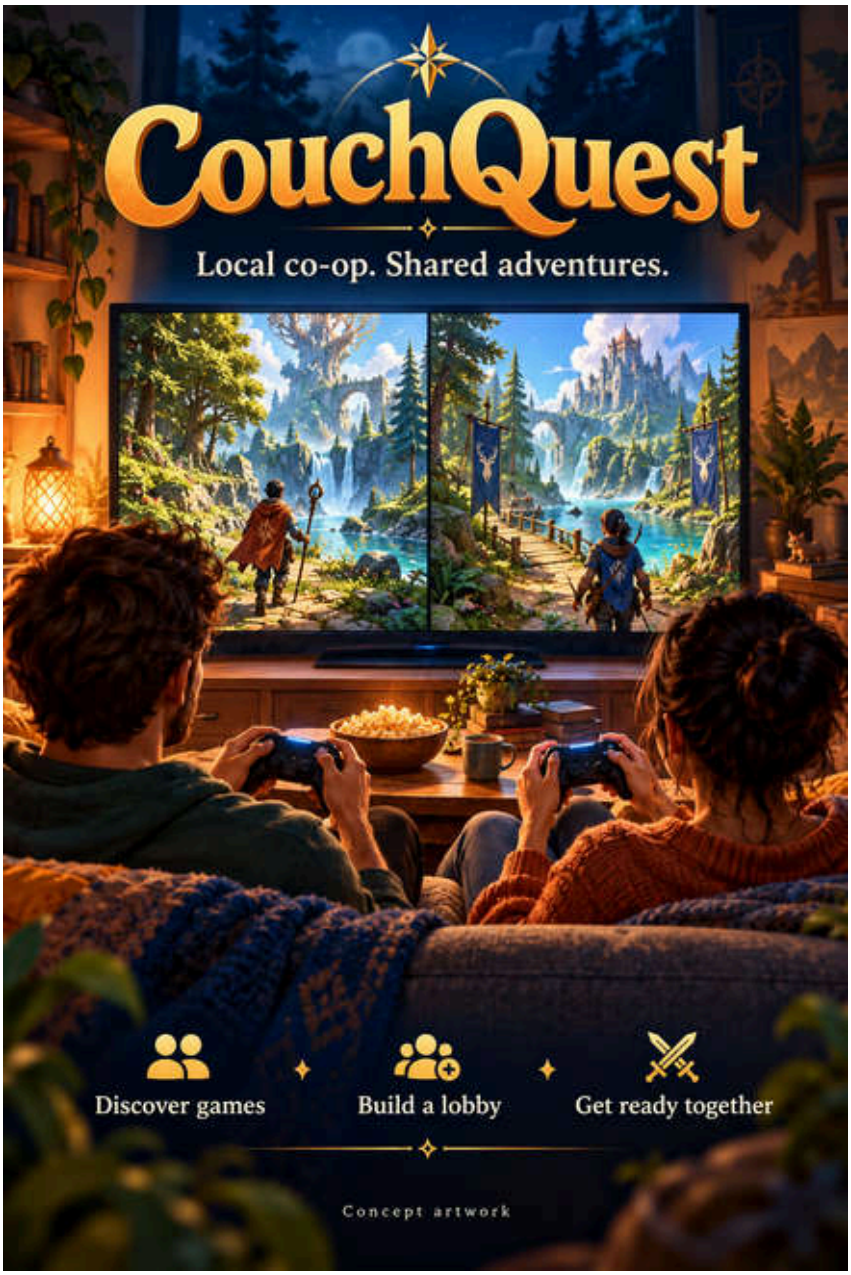
Title: CouchQuest Local co-op session platform

Assignment: Microservices Development and Deployment
SEZG583 Scalable Services

Chinar Goel, 2025MT03125



1 Work products and application access links 4
2 Architecture and business capabilities 5
3 System operations and collaboration 8
3a Core Scenarios 8
Catalog application 10
Play Sessions application 10
Core scenario 11
Twin Trails created 11
‘List Games’ showing Twin Trails 12
Alice creates the lobby 13
Premature start rejected 14
Bob joins the session 15
Eve cannot join a full session 16
Both players mark ready 17
Alice starts the session 19
Alice completes the session 20
3b CouchQuest API testing with Postman 21
Register the Twin Trails game 21
Find games for two players 22
Create session 22
Bob joins slot two 23
Validation rejects invalid actions 23
Alice marks herself ready 24
Bob marks himself ready 24
Alice starts the session 24
Alice completes the session 25
Read the completed session 25
4 Implementation and project structure 26
Data model 26
Lifecycle and business invariants 26
Scaling and resilience 27

5 Local execution and Docker deployment 28
Run services independently 28
Run Docker containers 28
6 Kubernetes deployment 29
Access and inspect 30
Scale independently 30
Local URLs 35
7 Scalability enhancement using HPA 36
HPA configuration 37
Scale up under application traffic 37
Automatic scale up recorded by Kubernetes 38
Automatic scale down after traffic stopped 38
Final healthy deployment 39
8 Repositories 40
9 CI and CD execution 42
Options considered 42
Independent releases 42

1 Work products and application access links

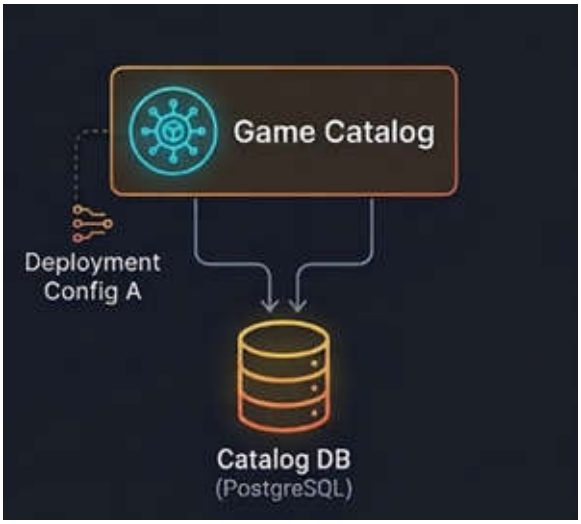
Work product	Full URL
Demonstration video	https://github.com/gchinar/CouchQuest/blob/main/CouchQuest_Demo_Chinar_%202025MT03125.mp4 Youtube link: https://youtu.be/-06LSeyG_Qc Drive link https://drive.google.com/file/d/14TSNzNpF7vuj9BaLDZSxv4kaBXsiavJ-/view?usp=sharing
Complete solution and integration repository	https://github.com/gchinar/CouchQuest
Independent Game Catalog service	https://github.com/gchinar/couchquest-catalog
Independent Play Session service	https://github.com/gchinar/couchquest-session
Postman collection	https://github.com/gchinar/CouchQuest/blob/main/postman_collection.json
Setup and execution instructions	https://github.com/gchinar/CouchQuest/blob/main/README.md
Catalog Dockerfile	https://github.com/gchinar/couchquest-catalog/blob/main/Dockerfile
Session Dockerfile	https://github.com/gchinar/couchquest-session/blob/main/Dockerfile
Catalog CI workflow	https://github.com/gchinar/couchquest-catalog/blob/main/.github/workflows/ci.yml
Session CI workflow	https://github.com/gchinar/couchquest-session/blob/main/.github/workflows/ci.yml

High Quality version of this document is at:

https://drive.google.com/file/d/1gKkrrETou9kLBuwpRhNV1ylgqN3LiKL_/view?usp=sharing

2 Architecture and business capabilities

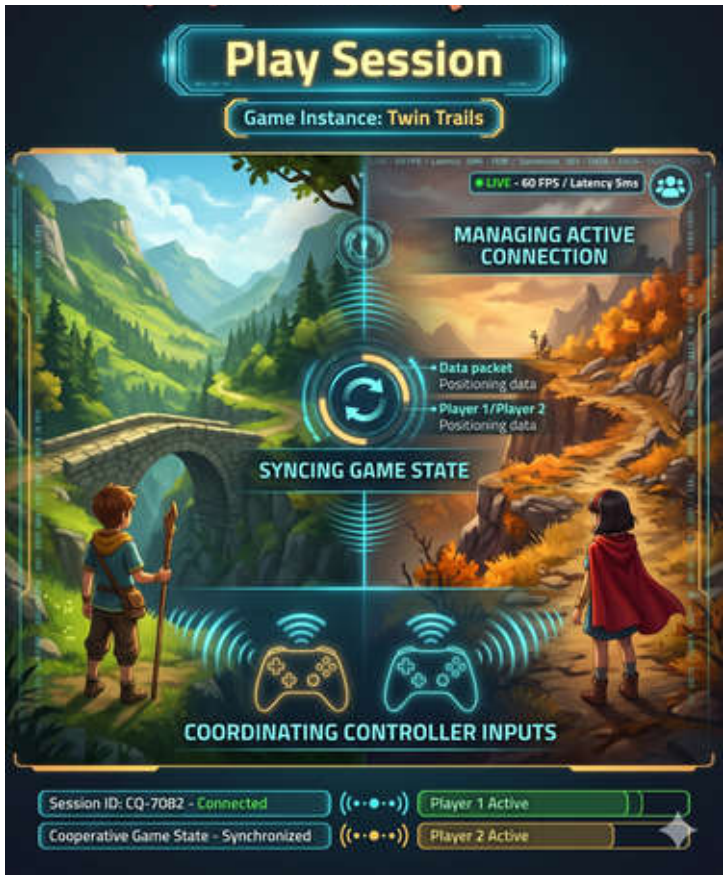
CouchQuest, a solution based on managing local split-screen co-op game sessions. CouchQuest coordinates local split-screen and shared-screen co-op sessions for two to four players. CouchQuest helps players discover local co-op games, prepare a lobby and get ready together. The solution implements Game Catalog and Play Session as independent microservices, each with its own database and deployment configuration.







Four original game concepts illustrate the catalog: Twin Trails and Orbit Racers use split-screen views; Kitchen Comets and Harbor Helpers use one shared view. Only Twin Trails is used in the implemented demonstration



Twin Trails is a proposed two-player forest puzzle adventure. Players follow separate trails, operate complementary mechanisms and reunite at a shared gate. Each player has a separate view on one display.

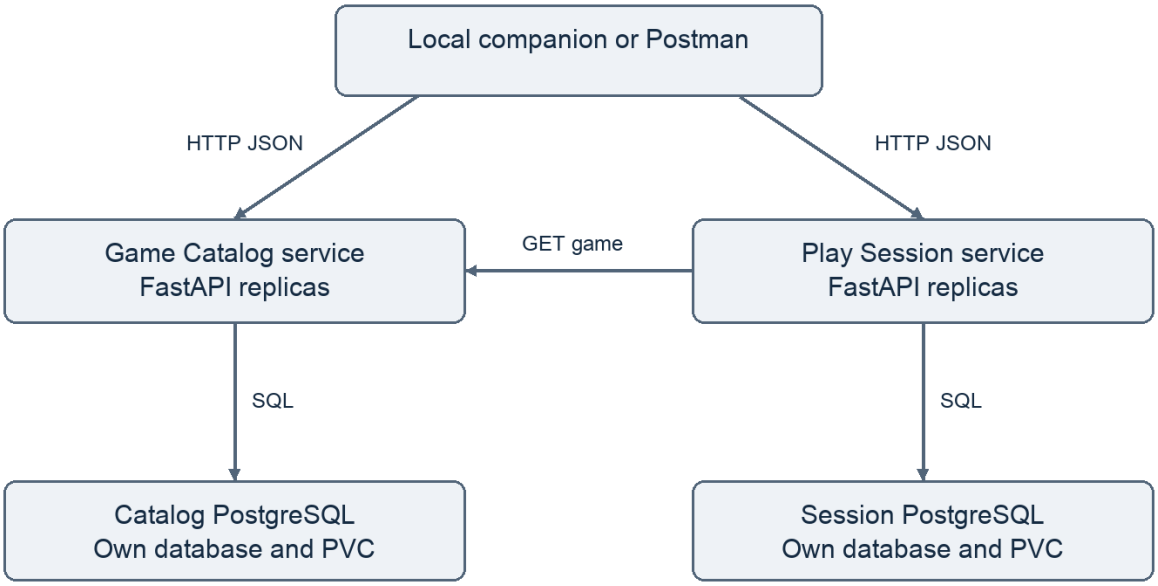


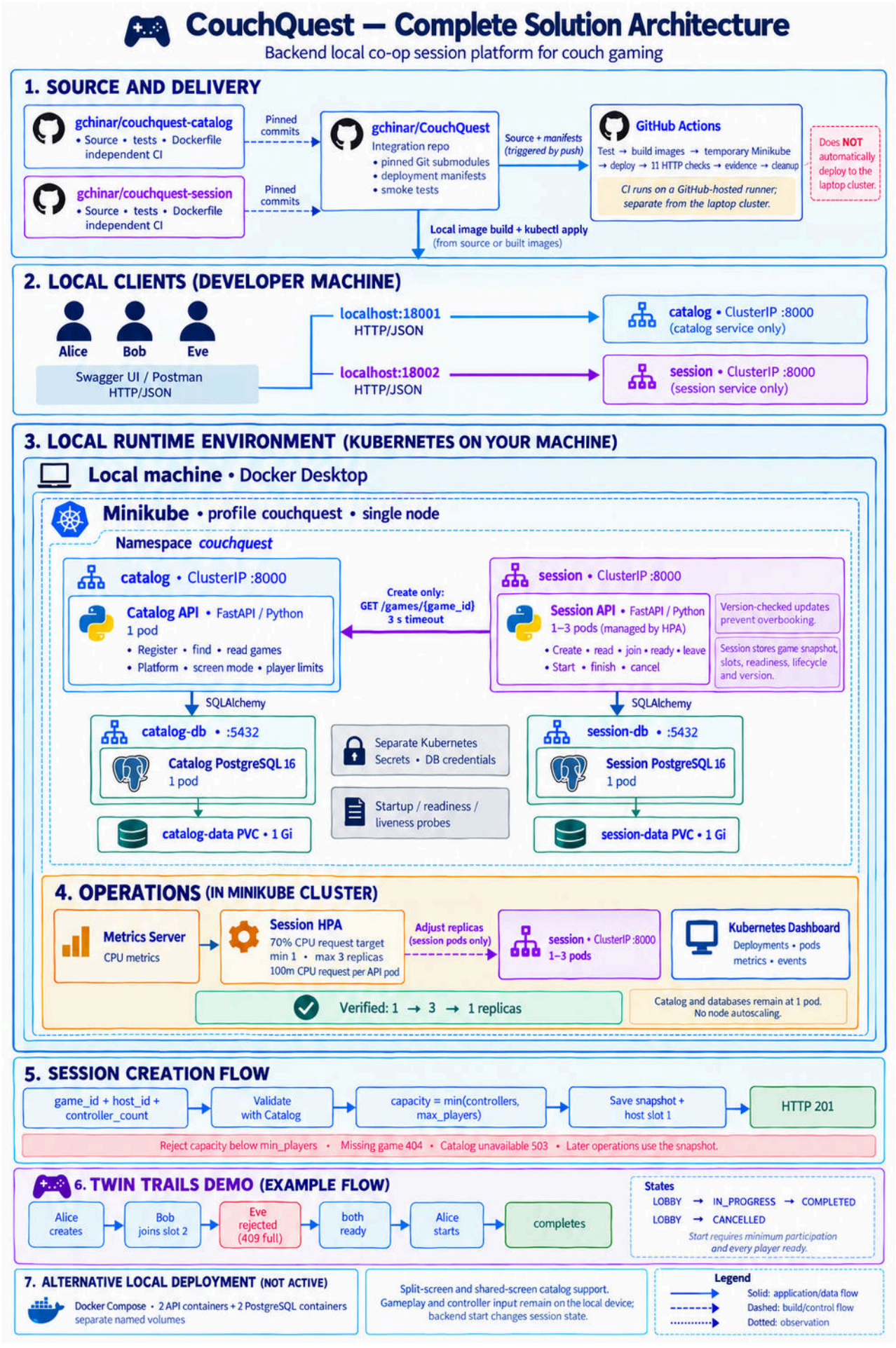
Figure 1 Implemented service collaboration and database ownership

Capability	Service and responsibility	Status
Compatibility and discovery	Catalog owns title, platform, display mode and player limits.	Implemented
Local session coordination	Session owns players, slots, readiness and lifecycle.	Implemented
Identity and access	Identity would verify player accounts and permissions.	Proposed
Results and achievements	Progress would persist game results and unlocks.	Proposed
Invitations and alerts	Notification would deliver invitations and completion alerts.	Proposed

Service boundaries follow business capabilities. Catalog changes need not be deployed with Session changes. There is no shared application database, shared table or cross-service SQL query. Each API can be built, tested and deployed independently.

Kubernetes provides an internal Service name for each API and database. Each API runs in its own Deployment. The teaching deployment uses one PostgreSQL instance and persistent volume per service. A future production gateway would provide TLS, authentication and rate limiting.

The accompanying package contains two service codebases, automated tests, Dockerfiles, separate PostgreSQL deployment configuration, Minikube manifests, a Postman collection and execution instructions. Identity, achievements and notifications are extension capabilities rather than implemented services.


CouchQuest — Complete Solution Architecture
Backend local co-op session platform for couch gaming
1. SOURCE AND DELIVERY
gchinar/couchquest-catalog
• Source • tests • Dockerfile
independent CI
Pinned commits
gchinar/couchquest-session
• Source • tests • Dockerfile
independent CI
Pinned commits
gchinar/CouchQuest
Integration repo
• pinned Git submodules
• deployment manifests
• smoke tests
Source + manifests (triggered by push)
GitHub Actions
Test → build images → temporary Minikube → deploy → 11 HTTP checks → evidence → cleanup
CI runs on a GitHub-hosted runner; separate from the laptop cluster.
Does NOT automatically deploy to the laptop cluster.
Local image build + kubectl apply
(from source or built images)
2. LOCAL CLIENTS (DEVELOPER MACHINE)
Alice
Bob
Eve
Swagger UI / Postman
HTTP/JSON
localhost:18001
HTTP/JSON
localhost:18002
HTTP/JSON
catalog • ClusterIP :8000
(catalog service only)
session • ClusterIP :8000
(session service only)
3. LOCAL RUNTIME ENVIRONMENT (KUBERNETES ON YOUR MACHINE)
Local machine • Docker Desktop
Minikube • profile couchquest • single node
Namespace couchquest
catalog • ClusterIP :8000
Catalog API • FastAPI / Python
1 pod
• Register • find • read games
• Platform • screen mode • player limits
Create only:
GET /games/{game_id}
3 s timeout
session • ClusterIP :8000
Session API • FastAPI / Python
1–3 pods (managed by HPA)
• Create • read • join • ready • leave
• Start • finish • cancel
Version-checked updates prevent overbooking.
Session stores game snapshot, slots, readiness, lifecycle and version.
SQLAlchemy
SQLAlchemy
catalog-db • :5432
Catalog PostgreSQL 16
1 pod
catalog-data PVC • 1 Gi
Separate Kubernetes Secrets • DB credentials
Startup / readiness / liveness probes
session-db • :5432
Session PostgreSQL 16
1 pod
session-data PVC • 1 Gi
4. OPERATIONS (IN MINIKUBE CLUSTER)
Metrics Server
CPU metrics
Session HPA
70% CPU request target
min 1 • max 3 replicas
100m CPU request per API pod
Adjust replicas (session pods only)
session • ClusterIP :8000
1–3 pods
Kubernetes Dashboard
Deployments • pods
metrics • events
Verified: 1 → 3 → 1 replicas
Catalog and databases remain at 1 pod.
No node autoscaling.
5. SESSION CREATION FLOW
game_id + host_id + controller_count
Validate with Catalog
capacity = min(controllers, max_players)
Save snapshot + host slot 1
HTTP 201
Reject capacity below min_players • Missing game 404 • Catalog unavailable 503 • Later operations use the snapshot.
6. TWIN TRAILS DEMO (EXAMPLE FLOW)
Alice creates
Bob joins slot 2
Eve rejected (409 full)
both ready
Alice starts
completes
States
LOBBY → IN_PROGRESS → COMPLETED
LOBBY → CANCELLED
Start requires minimum participation and every player ready.
7. ALTERNATIVE LOCAL DEPLOYMENT (NOT ACTIVE)
Docker Compose • 2 API containers + 2 PostgreSQL containers
separate named volumes
Split-screen and shared-screen catalog support.
Gameplay and controller input remain on the local device;
backend start changes session state.
Legend
Solid: application/data flow
Dashed: build/control flow
Dotted: observation

3 System operations and collaboration

3a Core Scenarios

Players use a local companion client or Postman to select a compatible game, create a lobby, allocate controller slots and mark themselves ready.

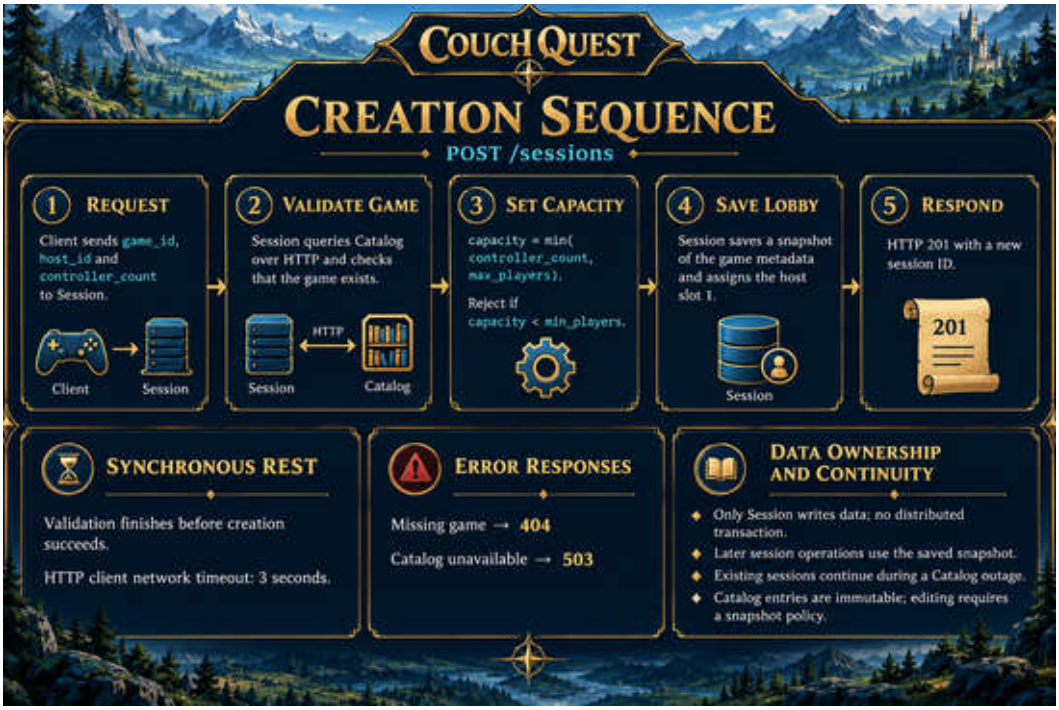
Commands change state; queries read it. The API separates these operations through HTTP methods while keeping a single service-owned data store per capability.

Type	Owner	Operation and route
Command	Catalog	Register game POST /games
Query	Catalog	Find games GET /games
Query	Catalog	Read game GET /games/{game_id}
Command	Session	Create lobby POST /sessions
Query	Session	Read lobby GET /sessions/{id}



Type	Owner	Operation and route
Command	Session	Join POST /sessions/{id}/players
Command	Session	Ready PUT /sessions/{id}/players/{player}/ready
Command	Session	Leave DELETE /sessions/{id}/players/{player}
Command	Session	Start POST /sessions/{id}/start
Command	Session	Finish POST /sessions/{id}/finish
Command	Session	Cancel POST /sessions/{id}/cancel

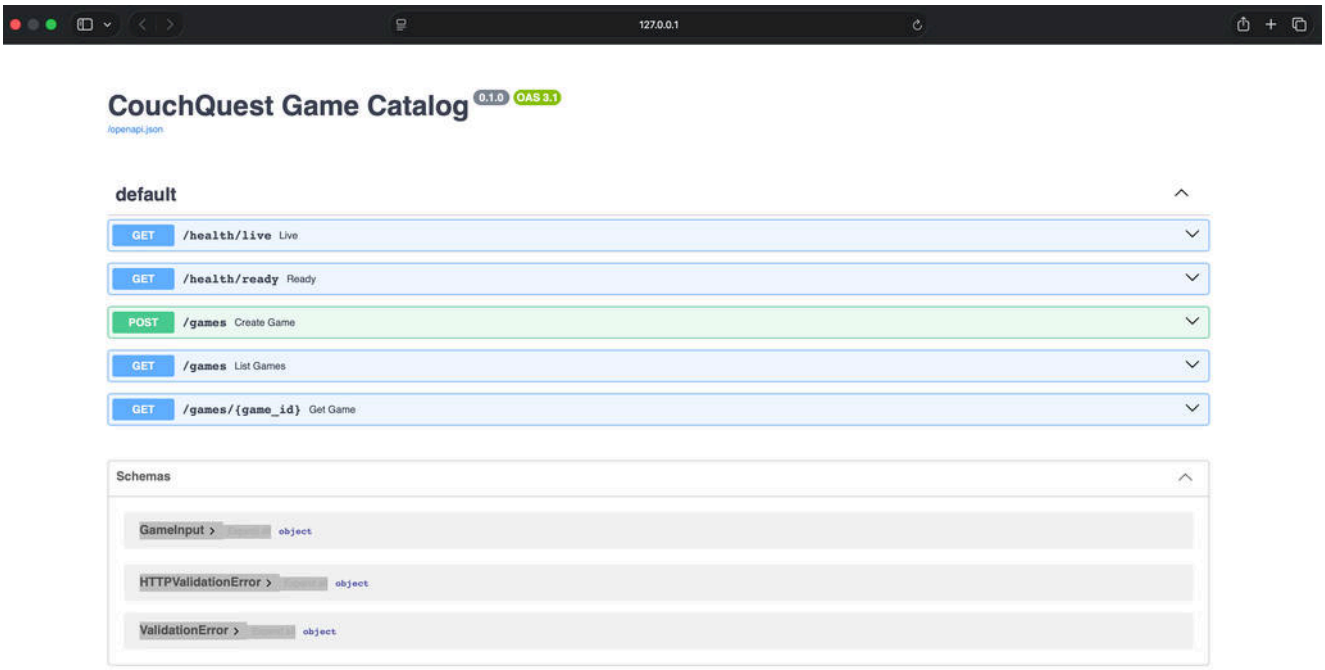




Catalog application

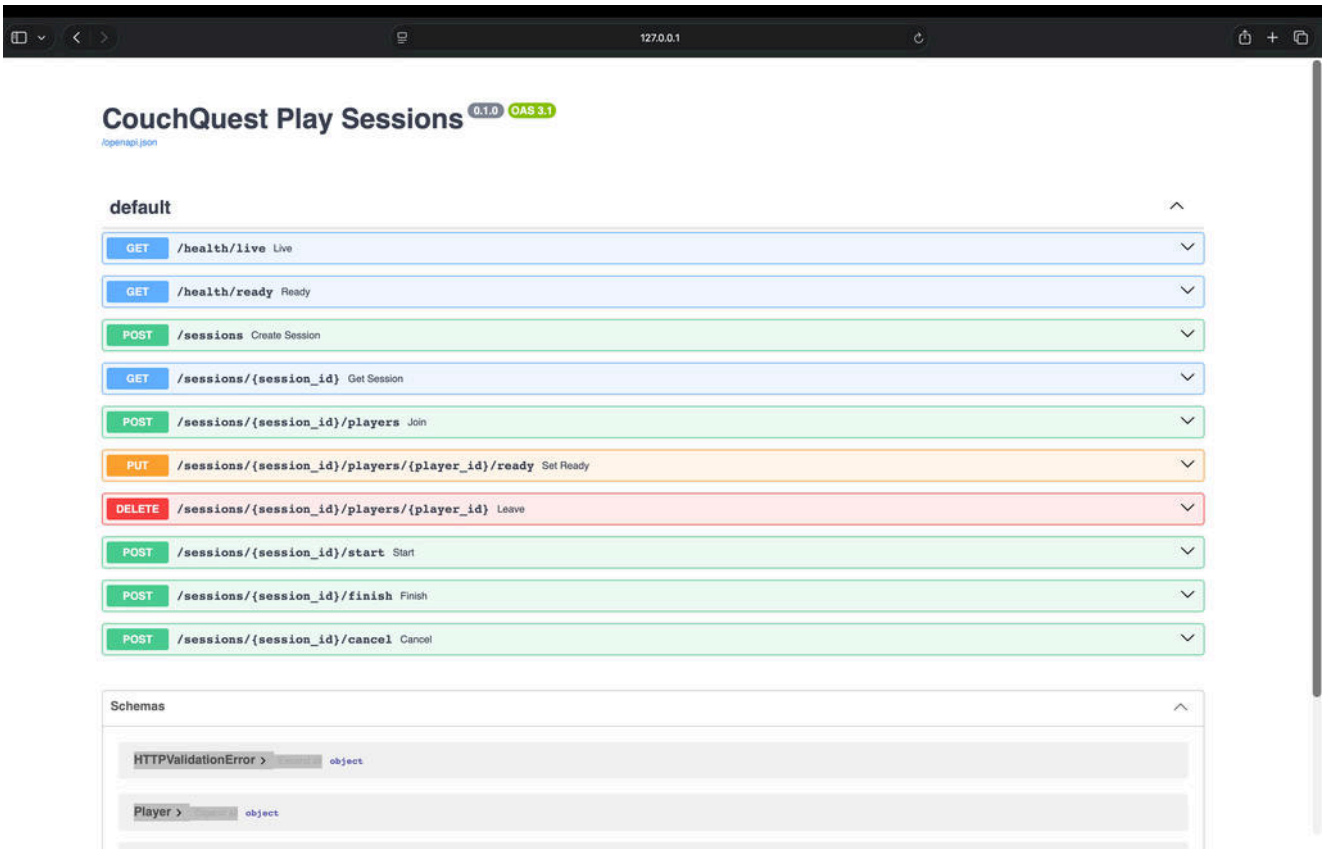
The running Catalog service exposes game creation, discovery and health operations.

The catalog represents two modes: Split-screen games which render a separate viewport for each player; shared-screen games which place all players in one common view.



Play Sessions application

The running Session service exposes lobby creation, joining, readiness, start and completion operations.



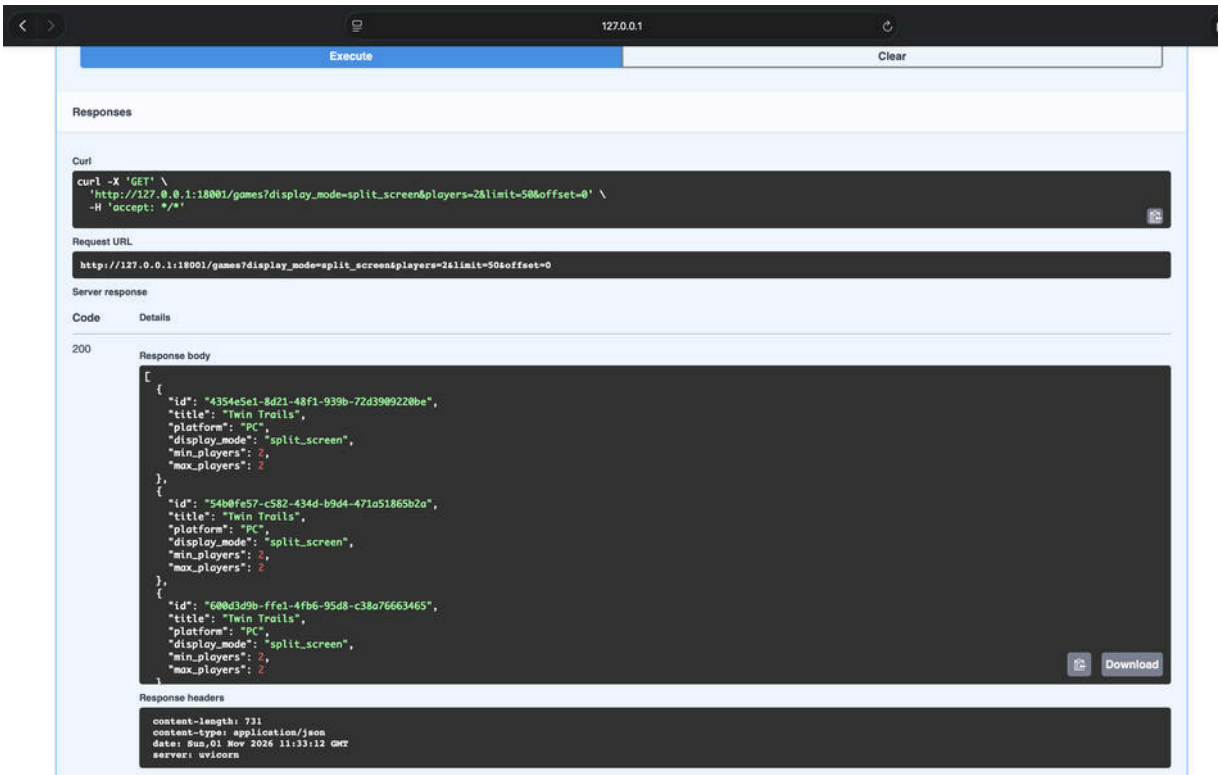
Core scenario

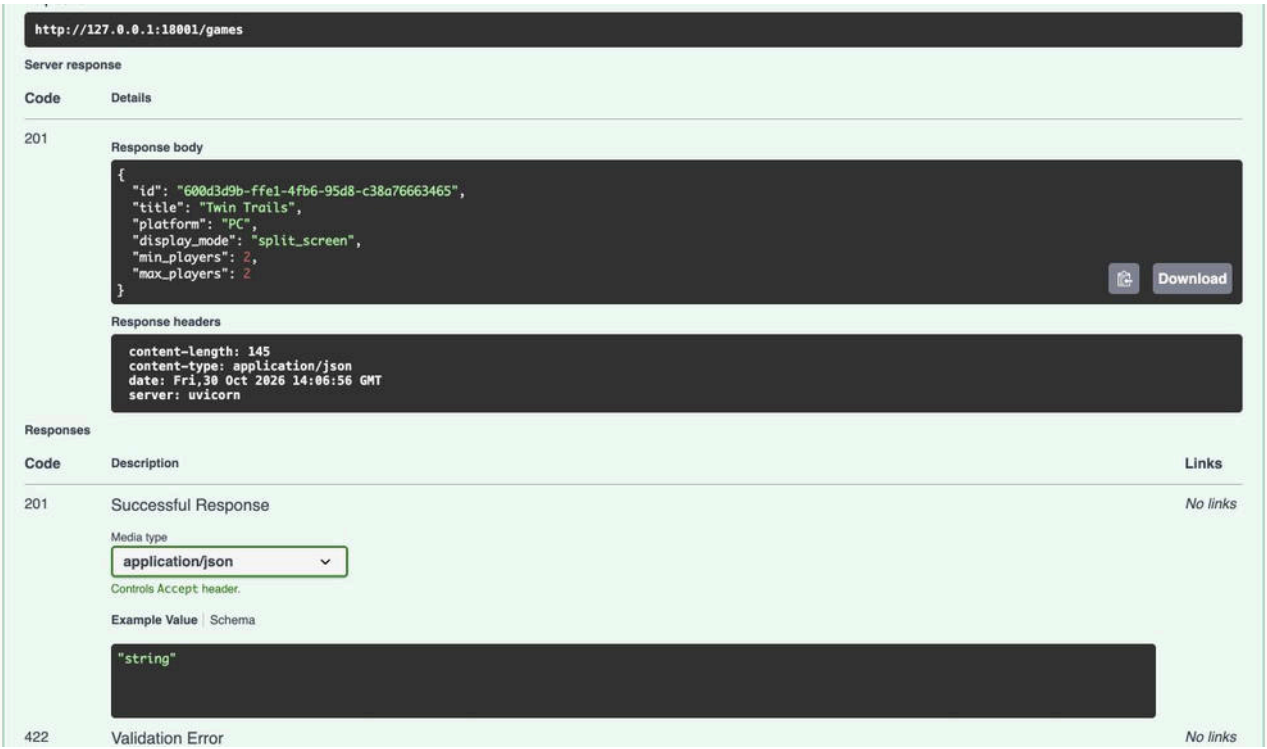
Alice creates a Twin Trails session with two controllers. Bob joins slot 2. An attempt by Eve to join is rejected because both slots are occupied. Alice and Bob mark themselves ready; Alice starts and later completes the session. The service rejects a start when minimum participation or readiness is missing.



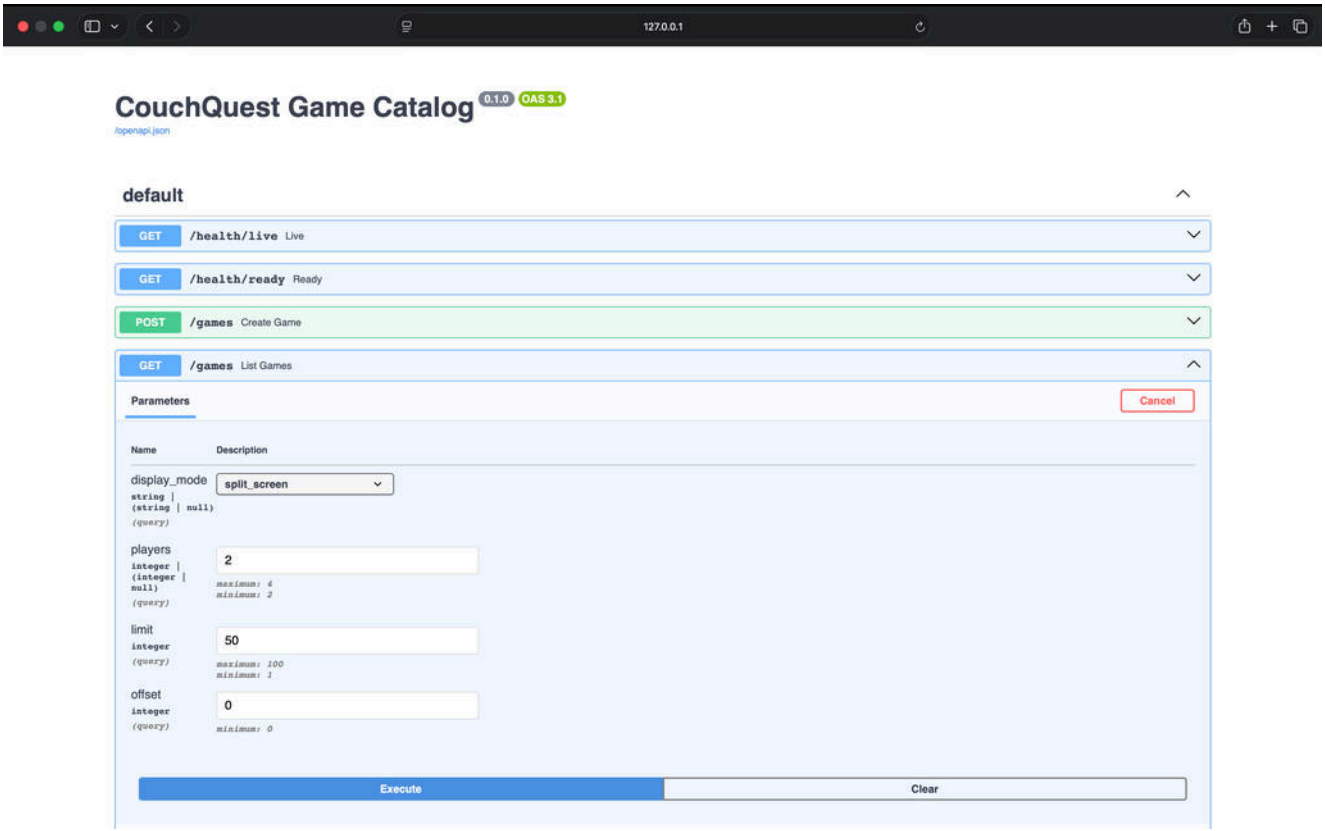
Twin Trails created

The actual POST /games response returns HTTP 201 and the two-player split-screen game record.





'List Games' showing Twin Trails



Alice creates the lobby

Alice creates a Twin Trails lobby with two controllers. The response is HTTP 201 with capacity 2 and status LOBBY.



Curl

```
curl -X 'POST' \
  'http://127.0.0.1:18002/sessions' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "game_id": "600d3d9b-ffe1-4fb6-95d8-c38a76663465",
  "host_id": "alice",
  "controller_count": 2
}'
```

Request URL

```
http://127.0.0.1:18002/sessions
```

Server response

Code | Details

201

Response body


```
{
  "id": "846f0350-f494-4031-912f-070a632bb2dd",
  "game": {
    "id": "600d3d9b-ffe1-4fb6-95d8-c38a76663465",
    "title": "Twin Trails",
    "platform": "PC",
    "display_mode": "split_screen",
    "min_players": 2,
    "max_players": 2
  },
  "host_id": "alice",
  "controller_count": 2,
  "capacity": 2,
  "status": "LOBBY",
  "players": [
    {
      "player_id": "alice",
      "slot": 1,
      "ready": false
    }
  ]
}
```


Download

Premature start rejected

A start attempt before the participation and readiness requirements are met returns HTTP 409.



Execute | Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:18002/sessions/846f0350-f494-4031-912f-070a632bb2dd/start' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{"player_id":"alice"}'
```

Request URL

```
http://127.0.0.1:18002/sessions/846f0350-f494-4031-912f-070a632bb2dd/start
```

Server response

Code	Details
409 *Undocumented*	Error: Conflict

Response body

```
{
  "detail": "Minimum players must join and every player must be ready"
}
```

Download

Response headers

```
content-length: 69
content-type: application/json
date: Fri,30 Oct 2026 14:07:59 GMT
server: uvicorn
```

Responses

Code	Description	Links



Bob joins the session

The join request for Bob succeeds with HTTP 200. The service assigns the available second slot.



Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:18002/sessions/846f0350-f494-4031-912f-070a632bb2dd/players' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{"player_id":"bob"}'
```

Request URL

```
http://127.0.0.1:18002/sessions/846f0350-f494-4031-912f-070a632bb2dd/players
```

Server response

Code	Details
200	Response body


```
{
  "id": "846f0350-f494-4031-912f-070a632bb2dd",
  "game": {
    "id": "600d3d9b-ffe1-4fb6-95d8-c38a76663465",
    "title": "Twin Trails",
    "platform": "PC",
    "display_mode": "split_screen",
    "min_players": 2,
    "max_players": 2
  },
  "host_id": "alice",
  "controller_count": 2,
  "capacity": 2,
  "status": "LOBBY",
  "players": [
    {
      "player_id": "alice",
      "slot": 1,
      "ready": false
    },
    {
      "player_id": "bob",
      "slot": 2,
```

Eve cannot join a full session

Eve's join attempt returns HTTP 409 with the response Session full.



Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:18002/sessions/846f0350-f494-4031-912f-070a632bb2dd/players' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{"player_id":"eve"}'
```

Request URL

```
http://127.0.0.1:18002/sessions/846f0350-f494-4031-912f-070a632bb2dd/players
```

Server response

Code	Details
409 *Undocumented*	Error: Conflict

Response body

```
{
  "detail": "Session full"
}
```

Download

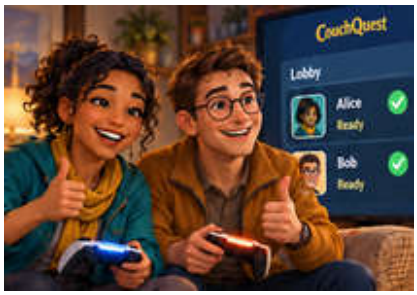
Response headers

```
content-length: 25
content-type: application/json
date: Fri,30 Oct 2026 14:08:18 GMT
server: uvicorn
```

Responses

Code	Description	Links

Both players mark ready



The readiness update for Alice succeeds with HTTP 200 while the session remains in LOBBY.

Curl

```
curl -X 'PUT' \
  'http://127.0.0.1:18002/sessions/846f0350-f494-4031-912f-070a632bb2dd/players/alice/ready' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "ready": true
}'
```

Request URL

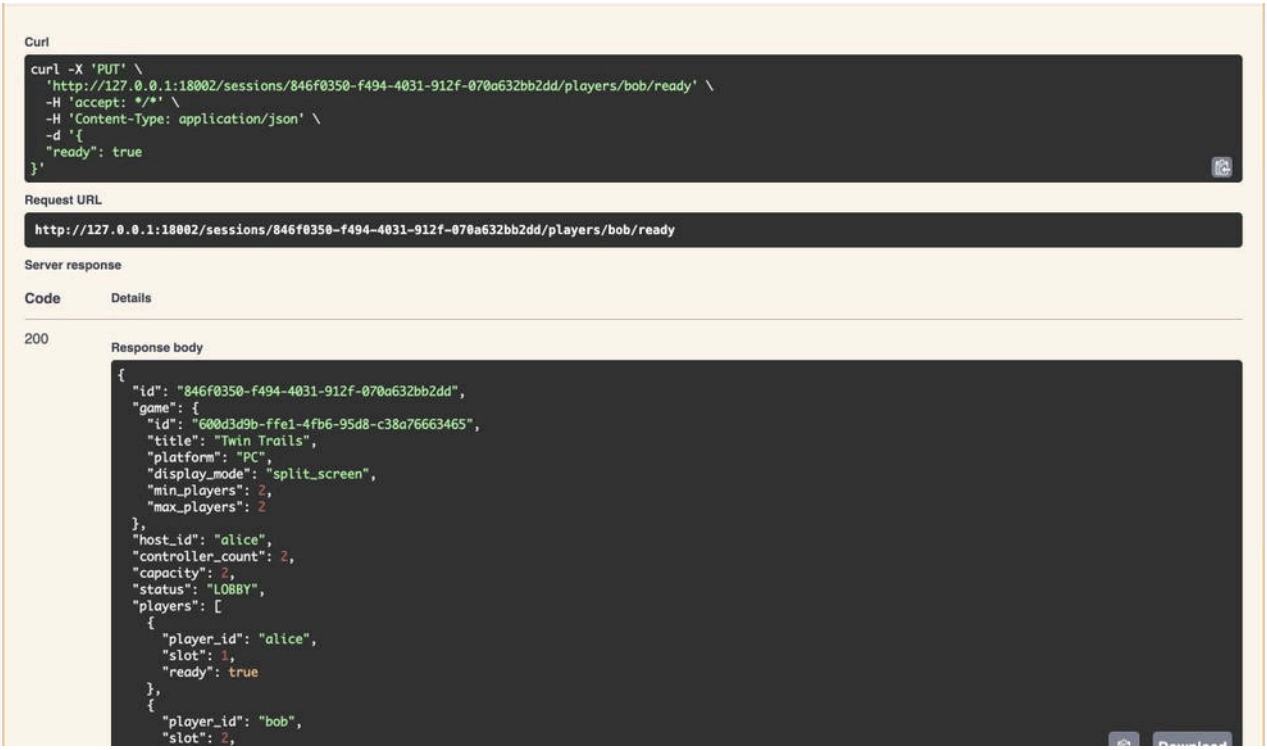
```
http://127.0.0.1:18002/sessions/846f0350-f494-4031-912f-070a632bb2dd/players/alice/ready
```

Server response

Code	Details
200	Response body

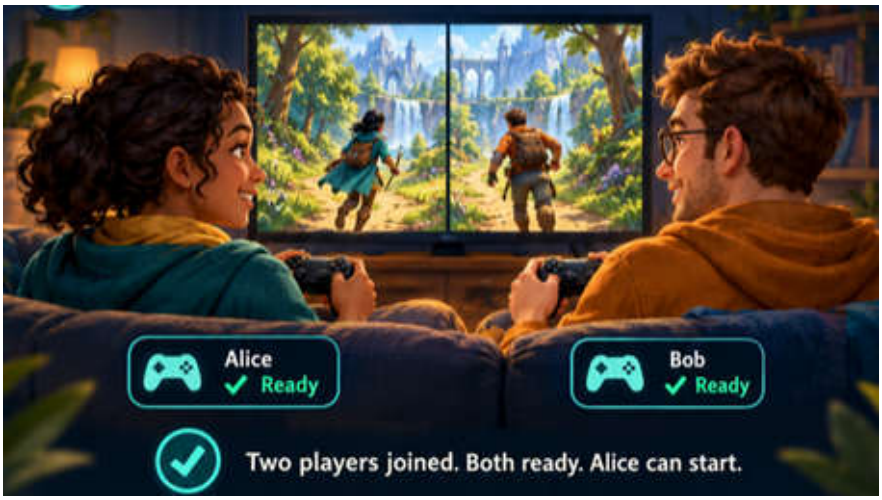

```
{
  "id": "846f0350-f494-4031-912f-070a632bb2dd",
  "game": {
    "id": "600d3d9b-ffe1-4fb6-95d8-c38a76663465",
    "title": "Twin Trails",
    "platform": "PC",
    "display_mode": "split_screen",
    "min_players": 2,
    "max_players": 2
  },
  "host_id": "alice",
  "controller_count": 2,
  "capacity": 2,
  "status": "LOBBY",
  "players": [
    {
      "player_id": "alice",
      "slot": 1,
      "ready": true
    },
    {
      "player_id": "bob",
      "slot": 2,
```


Bob’s readiness update succeeds with HTTP 200. Both participants are now ready.


Curl
curl -X 'PUT' \
'http://127.0.0.1:18002/sessions/846f0350-f494-4031-912f-070a632bb2dd/players/bob/ready' \
-H 'accept: */*' \
-H 'Content-Type: application/json' \
-d '{
"ready": true
}'
Request URL
http://127.0.0.1:18002/sessions/846f0350-f494-4031-912f-070a632bb2dd/players/bob/ready
Server response
Code
Details
200
Response body
{
"id": "846f0350-f494-4031-912f-070a632bb2dd",
"game": {
"id": "600d3d9b-ffe1-4fb6-95d8-c38a76663465",
"title": "Twin Trails",
"platform": "PC",
"display_mode": "split_screen",
"min_players": 2,
"max_players": 2
},
"host_id": "alice",
"controller_count": 2,
"capacity": 2,
"status": "LOBBY",
"players": [
{
"player_id": "alice",
"slot": 1,
"ready": true
},
{
"player_id": "bob",
"slot": 2,

Alice starts the session

The start request succeeds with HTTP 200 and changes the session to IN_PROGRESS.



Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:18002/sessions/846f0350-f494-4031-912f-070a632bb2dd/start' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{"player_id":"alice"}'
```

Request URL

```
http://127.0.0.1:18002/sessions/846f0350-f494-4031-912f-070a632bb2dd/start
```

Server response

Code | Details

200

Response body


```
{
  "id": "846f0350-f494-4031-912f-070a632bb2dd",
  "game": {
    "id": "600d3d9b-ffe1-4fb6-95d8-c38a76663465",
    "title": "Twin Trails",
    "platform": "PC",
    "display_mode": "split_screen",
    "min_players": 2,
    "max_players": 2
  },
  "host_id": "alice",
  "controller_count": 2,
  "capacity": 2,
  "status": "IN_PROGRESS",
  "players": [
    {
      "player_id": "alice",
      "slot": 1,
      "ready": true
    },
    {
      "player_id": "bob",
      "slot": 2,
```

Alice completes the session

The finish request succeeds with HTTP 200 and changes the session to COMPLETED.



Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:18002/sessions/846f0350-f494-4031-912f-070a632bb2dd/finish' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{"player_id":"alice"}'
```

Request URL

```
http://127.0.0.1:18002/sessions/846f0350-f494-4031-912f-070a632bb2dd/finish
```

Server response

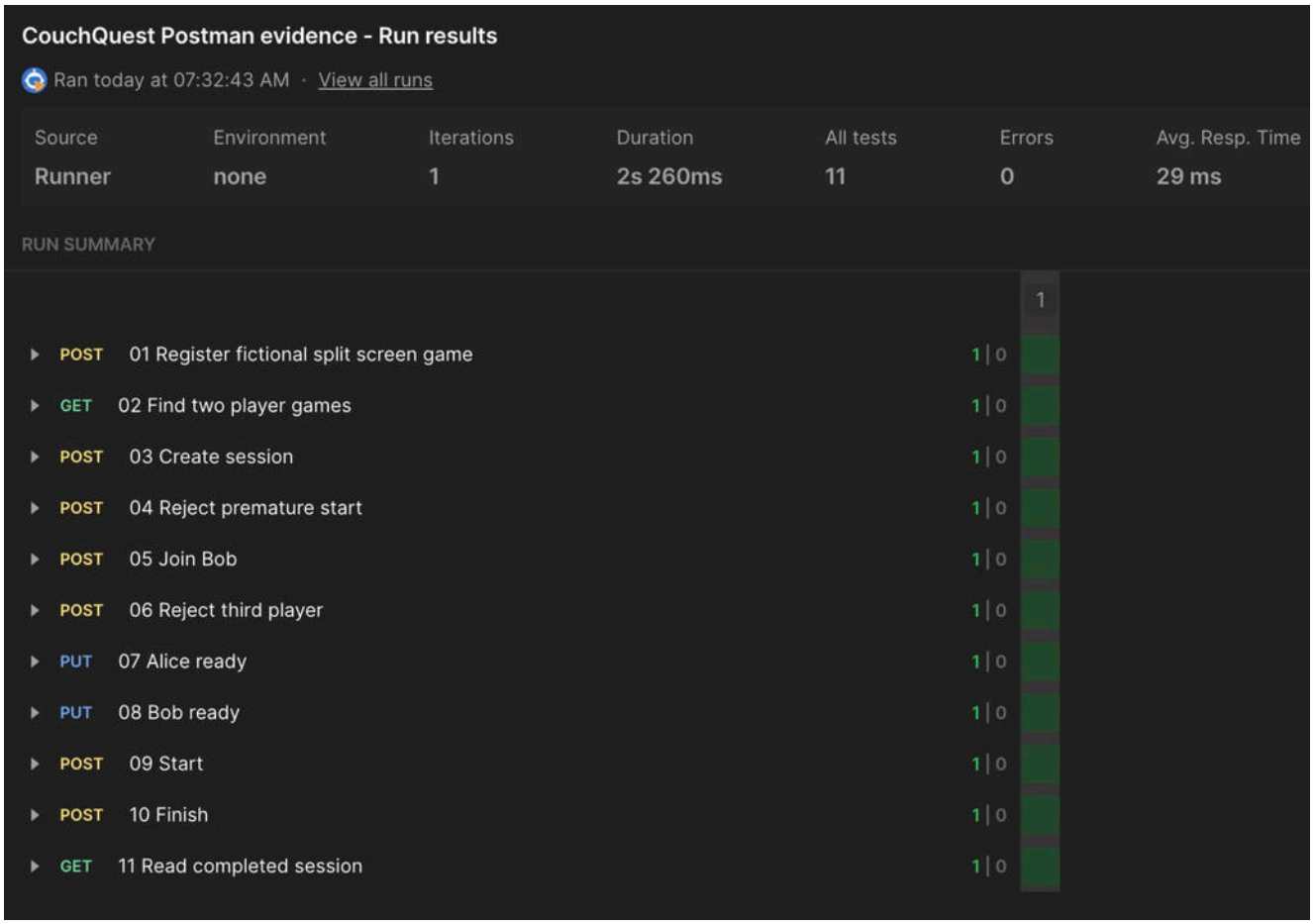
Code	Details
200	Response body


```
{
  "id": "846f0350-f494-4031-912f-070a632bb2dd",
  "game": {
    "id": "600d3d9b-ffe1-4fb6-95d8-c38a76663465",
    "title": "Twin Trails",
    "platform": "PC",
    "display_mode": "split_screen",
    "min_players": 2,
    "max_players": 2
  },
  "host_id": "alice",
  "controller_count": 2,
  "capacity": 2,
  "status": "COMPLETED",
  "players": [
    {
      "player_id": "alice",
      "slot": 1,
      "ready": true
    },
    {
      "player_id": "bob",
      "slot": 2,
```

3b CouchQuest API testing with Postman

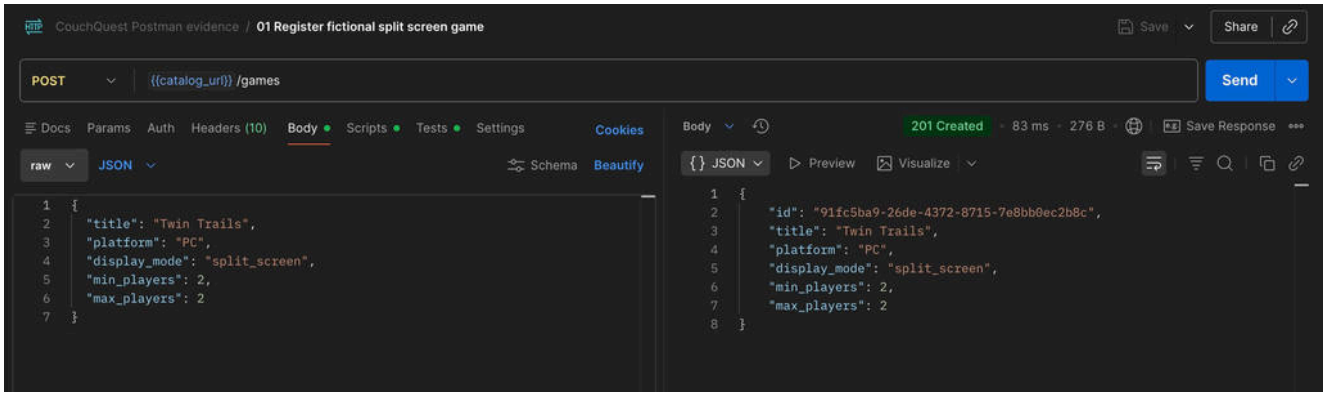
Postman Desktop 11.77.2 was used to test the Catalog and Session services running in the local CouchQuest Minikube cluster. The screenshots below record real requests and responses, including rejected actions.

Catalog base URL: http://127.0.0.1:18001 | Session base URL: http://127.0.0.1:18002
The requests use collection variables catalog_url and session_url. Scripts save game_id and session_id from successful creation responses.



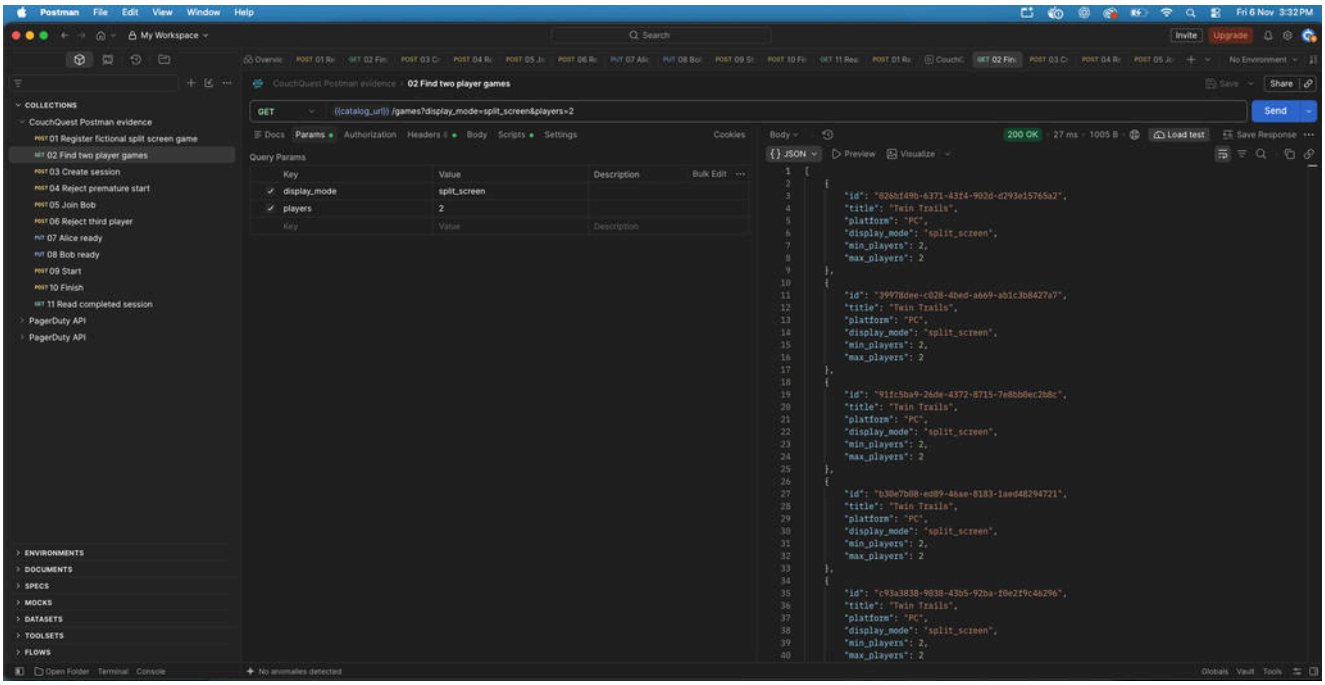
Register the Twin Trails game

POST /games returns HTTP 201 with a game ID. Twin Trails is a PC split-screen game requiring exactly two players. The collection saves the returned ID for session creation.



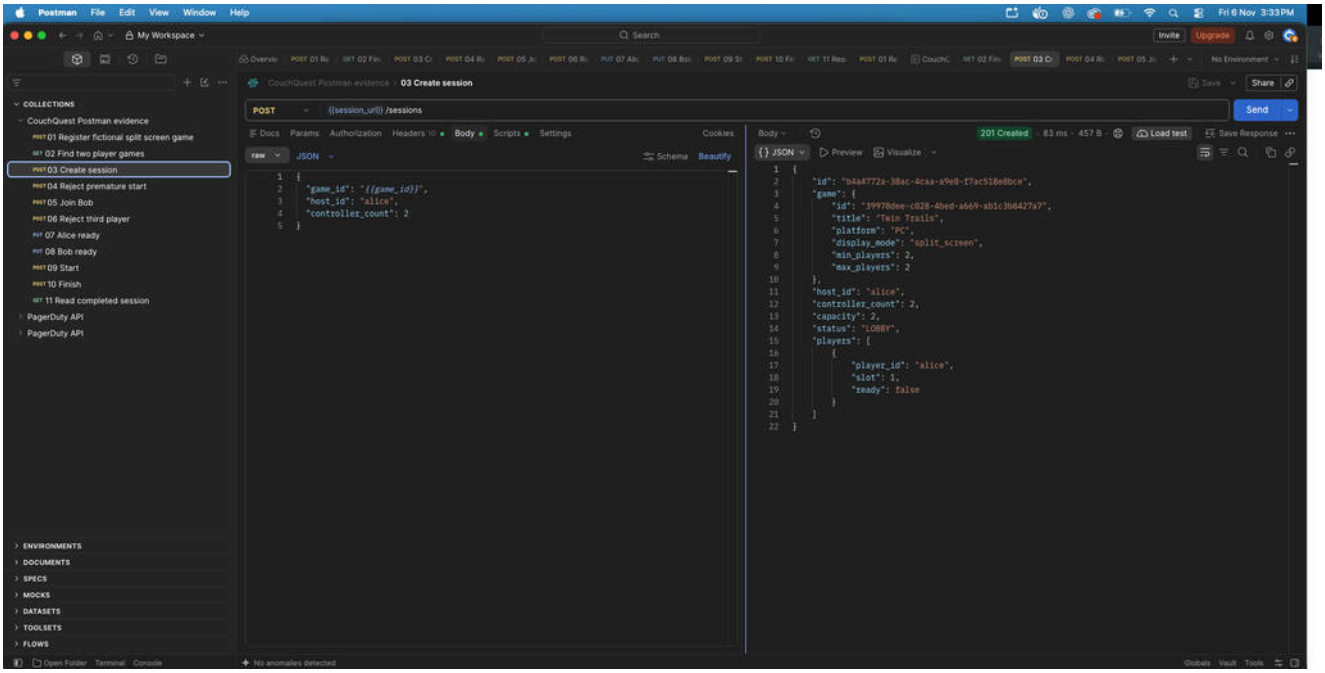
Find games for two players

GET /games?display_mode=split_screen&players=2 returns HTTP 200 and matching game metadata. The query filters by display mode and supported player count.Create a lobby for Alice



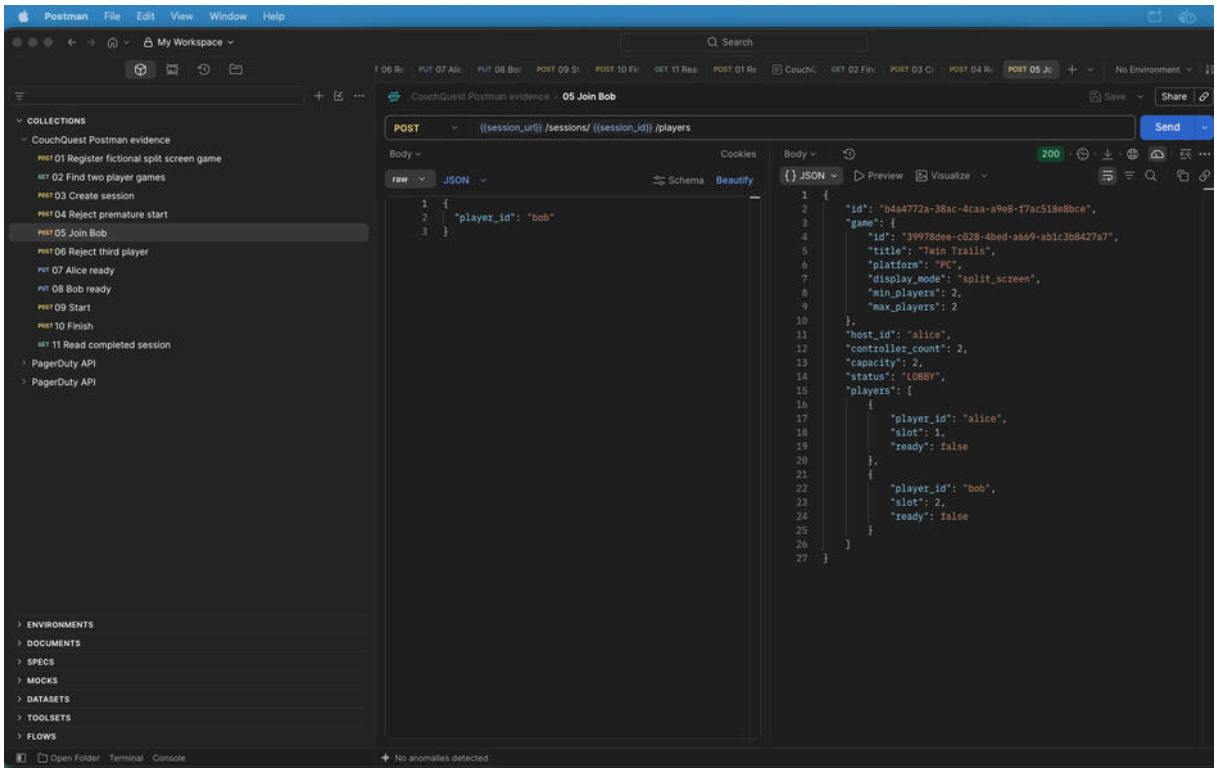
Create session

POST /sessions sends the saved game_id, host_id alice and controller_count 2. HTTP 201 confirms a new LOBBY with capacity 2 and Alice assigned slot 1. The response includes the saved game metadata.



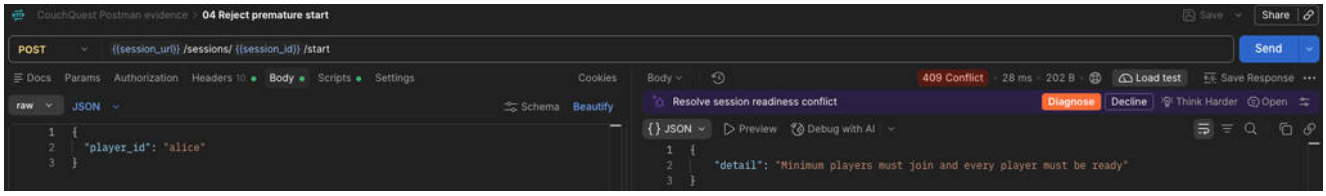
Bob joins slot two

POST /sessions/{session_id}/players with player_id bob returns HTTP 200. Alice remains in slot 1 and Bob occupies slot 2. Both players initially have ready set to false.Validation rejects invalid actions

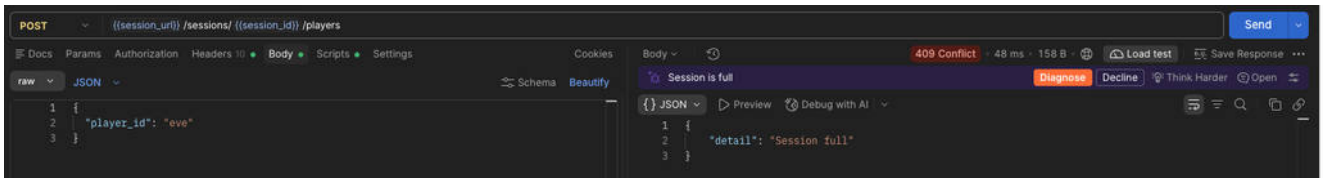


Validation rejects invalid actions

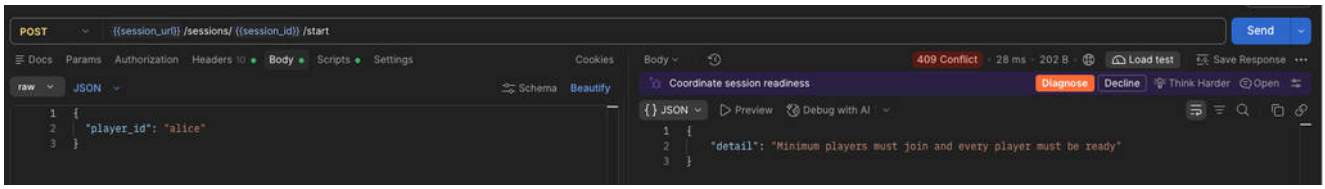
Before Bob joins, start returns HTTP 409 because participation and readiness are incomplete.



After Bob joins, Eve receives HTTP 409 with Session full. Capacity remains two.

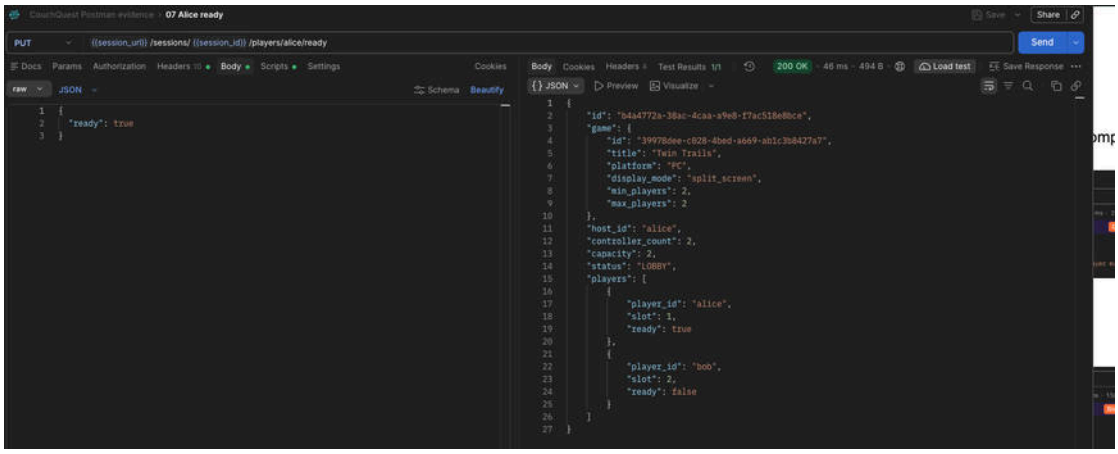


With both players present but not ready, another start still returns HTTP 409.



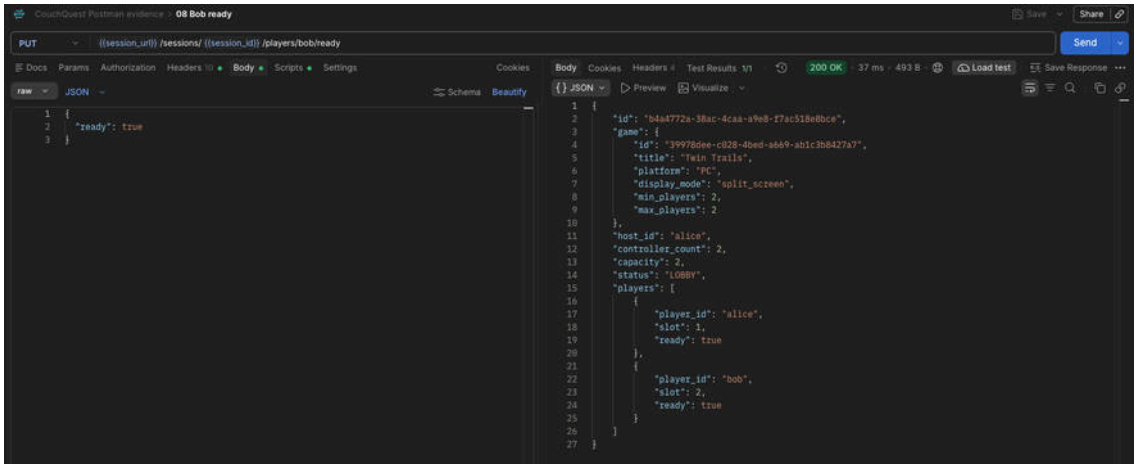
Alice marks herself ready

PUT /sessions/{session_id}/players/alice/ready with ready true returns HTTP 200. Alice is ready while Bob is still not ready, and the session remains in LOBBY.Bob marks himself ready



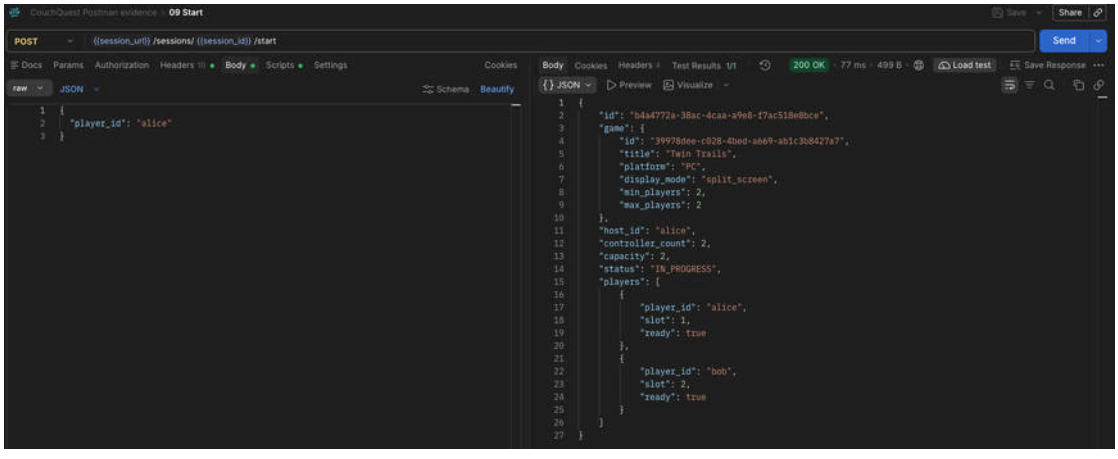
Bob marks himself ready

PUT /sessions/{session_id}/players/bob/ready returns HTTP 200. Both Alice and Bob now have ready true. Both player records are visible in the response.



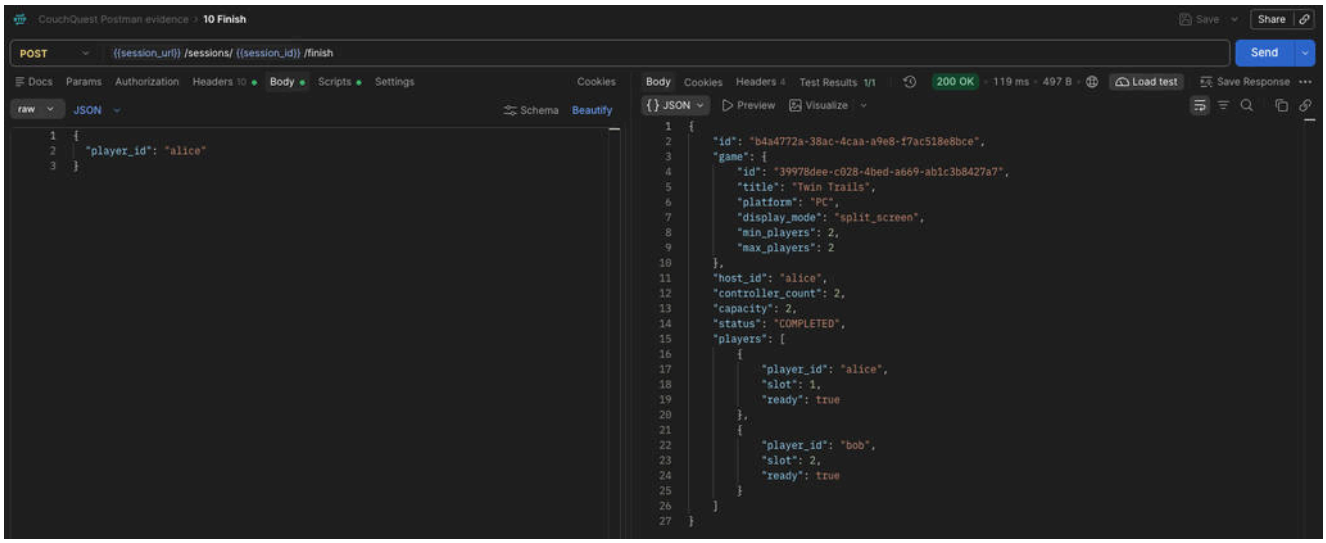
Alice starts the session

POST /sessions/{session_id}/start with player_id alice returns HTTP 200. After the participation and readiness checks succeed, status changes to IN_PROGRESS.Alice completes the session



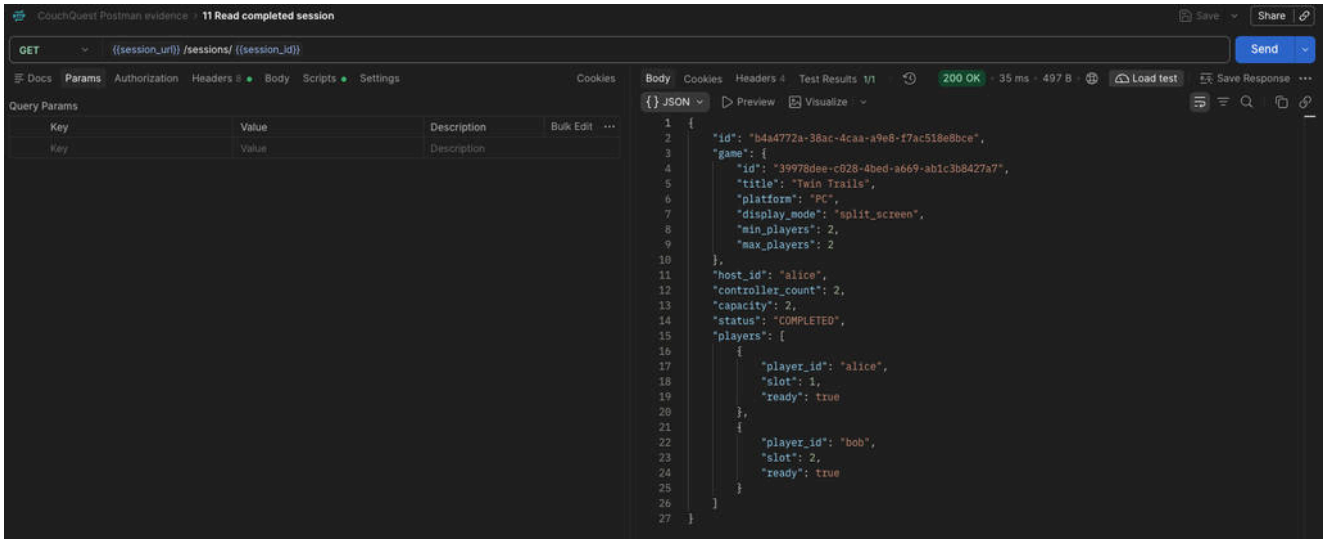
Alice completes the session

POST /sessions/{session_id}/finish with player_id alice returns HTTP 200 and status COMPLETED. The response retains both player records and the saved game metadata.



Read the completed session

GET /sessions/{session_id} returns HTTP 200 and the persisted COMPLETED state. This final query confirms the result can be retrieved independently after the finish command.



4 Implementation and project structure

CouchQuest ran in four separate Docker containers and on a local Minikube Kubernetes cluster. Each environment passed 11 HTTP checks, including expected business-rule rejections and the final COMPLETED state. Docker Engine 29.4.3, Minikube 1.39.0 and Kubernetes 1.35.0 were used; the local cluster had 8 CPUs and 4000 MB memory.

All four deployments are available: Catalog 1/1, Session 2/2 and each database 1/1. Session was scaled independently. Four ClusterIP Services expose the APIs internally on port 8000 and databases on 5432. Each database has its own Bound 1 GiB PVC. A completed session was retrieved successfully after a rolling Session API restart.

Technology	Purpose
Python 3.12 and FastAPI	Independent REST APIs and generated API documentation
Pydantic	Input validation and player/controller bounds
SQLAlchemy and PostgreSQL 16	Persistence with one database per service
HTTPX	Session-to-Catalog HTTP communication
pytest	Business-rule, failure and concurrency tests
Docker and Kubernetes	Container packaging, service discovery and deployment

Data model

Store	Fields
Catalog games	id, title, platform, display_mode, min_players, max_players
Session sessions	id, version, data JSON
Session JSON content	Game snapshot, host ID, controller count, capacity, status, and players with player ID, slot and ready flag

Lifecycle and business invariants

Valid transitions are LOBBY → IN_PROGRESS → COMPLETED, or LOBBY → CANCELLED. Joining, leaving and setting readiness require LOBBY. Starting requires the minimum player count and every participant ready. The host must cancel instead of leaving. Only the supplied host ID may start, finish or cancel.

Duplicate joins do not allocate another slot. Repeated readiness writes set the same value. Repeating start while already in progress or finish after completion succeeds with the current state.

Scaling and resilience

API replicas keep durable state in their own PostgreSQL database. Catalog and Session can scale independently. The HPA targets 70 percent CPU utilization with one to three replicas. Readiness checks database access; liveness checks the process. Resource requests and limits are included. Persistent volumes preserve data when API pods restart.

```
catalog-service/                 session-service/
  app/main.py                      app/main.py
  tests/test_api.py                tests/test_api.py
  Dockerfile                       Dockerfile
  requirements.txt                 requirements.txt
  requirements-dev.txt             requirements-dev.txt
  .github/workflows/ci.yml          .github/workflows/ci.yml
  README.md                        README.md

deployment/
  compose.yaml   kubernetes.yaml   hpa.yaml   .env.example
postman_collection.json   smoke_test.py   evidence/
```

Each service is maintained in a separate GitHub repository with its own dependencies, tests, Dockerfile, CI and Kubernetes manifests. CouchQuest pins both repositories as Git submodules.

5 Local execution and Docker deployment

Run services independently

Requires Python 3.12. In each service directory, create its own virtual environment, install dependencies and run the tests. Start the APIs in separate terminals.

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements-dev.txt
.venv/bin/python -m pytest -q

# Catalog terminal from catalog-service
.venv/bin/python -m uvicorn app.main:app --port 8001

# Session terminal from session-service
.venv/bin/python -m uvicorn app.main:app --port 8002
```

Local defaults use separate SQLite files. The Session service calls http://localhost:8001. Interactive API documentation is available at http://localhost:8001/docs and http://localhost:8002/docs. From the package root, run python3 smoke_test.py.

Run Docker containers

Start the Docker engine. From the package root, copy the example environment file and replace its passwords with URL-safe alphanumeric values. The environment file must not be committed.

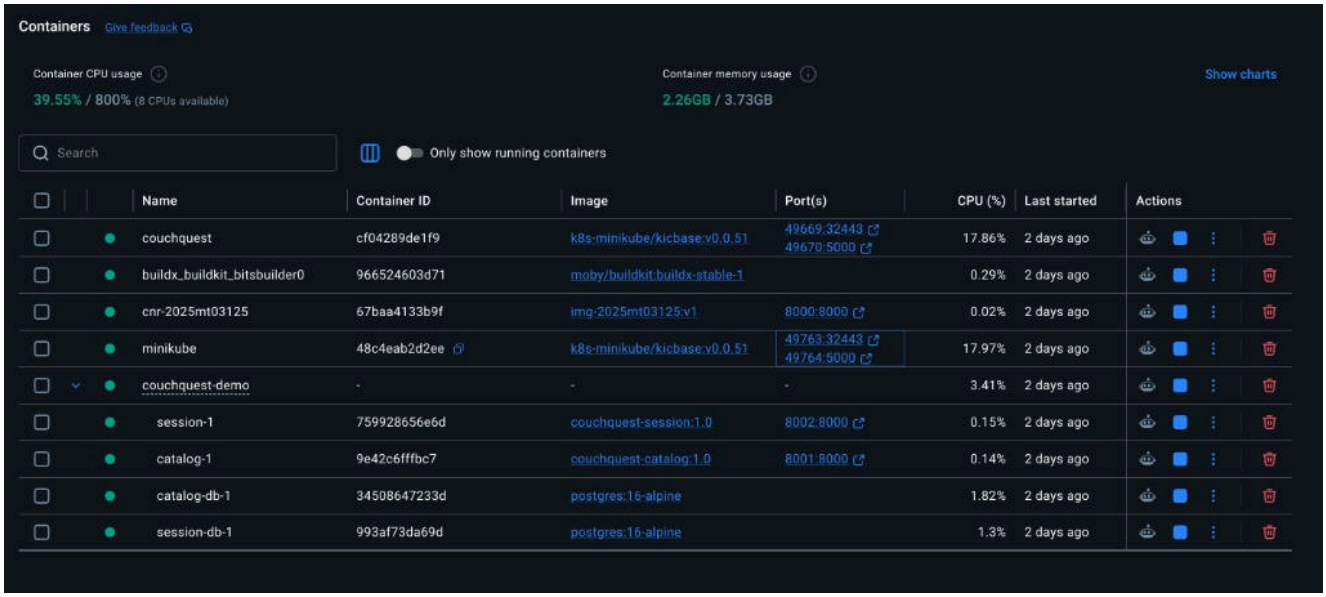
```
cp deployment/.env.example deployment/.env

docker compose --env-file deployment/.env \
  -f deployment/compose.yaml up --build -d

docker compose --env-file deployment/.env \
  -f deployment/compose.yaml ps

python3 smoke_test.py
```

The Compose deployment starts four containers: two APIs and two PostgreSQL databases. Database health checks gate API startup. Each database uses its own named volume. API host ports bind to 127.0.0.1, and the databases have no published host ports.



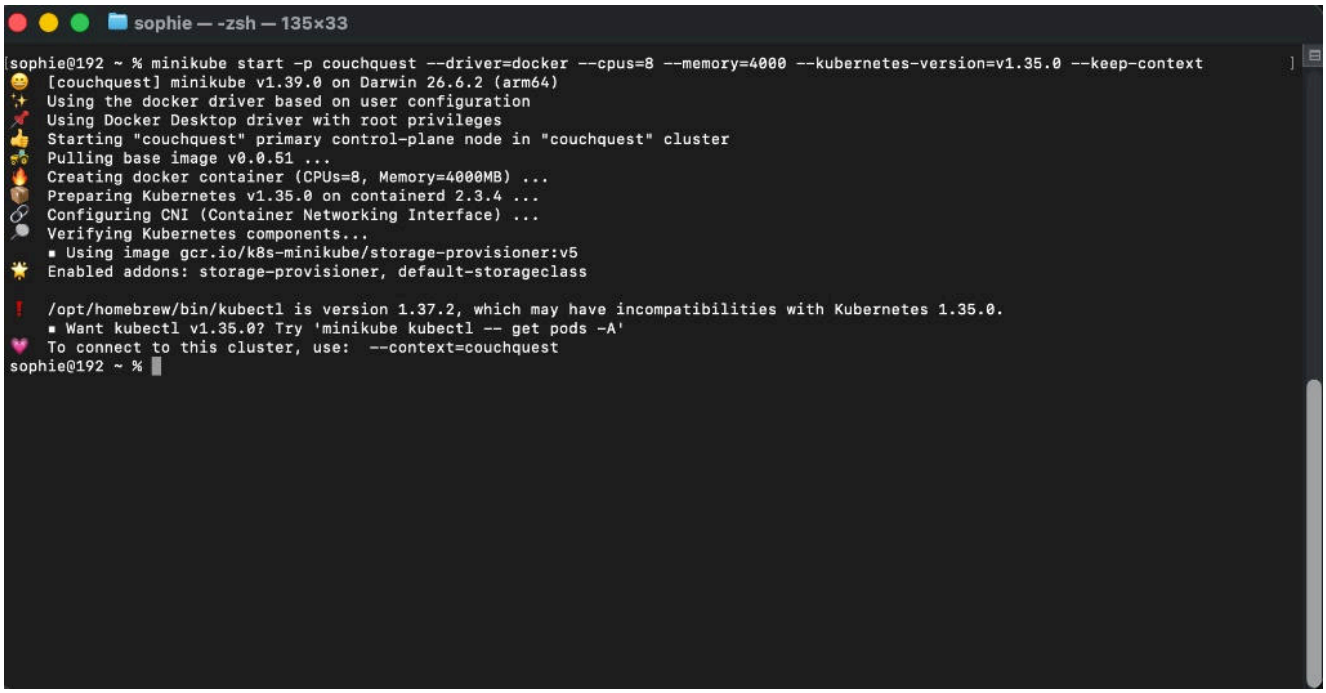
6 Kubernetes deployment

Use Docker and the dedicated couchquest Minikube profile. The commands preserve the current default Kubernetes context. Full instructions are in README.md.

```
minikube start -p couchquest --driver=docker \
  --cpus=8 --memory=4000 --kubernetes-version=v1.35.0 \
  --keep-context

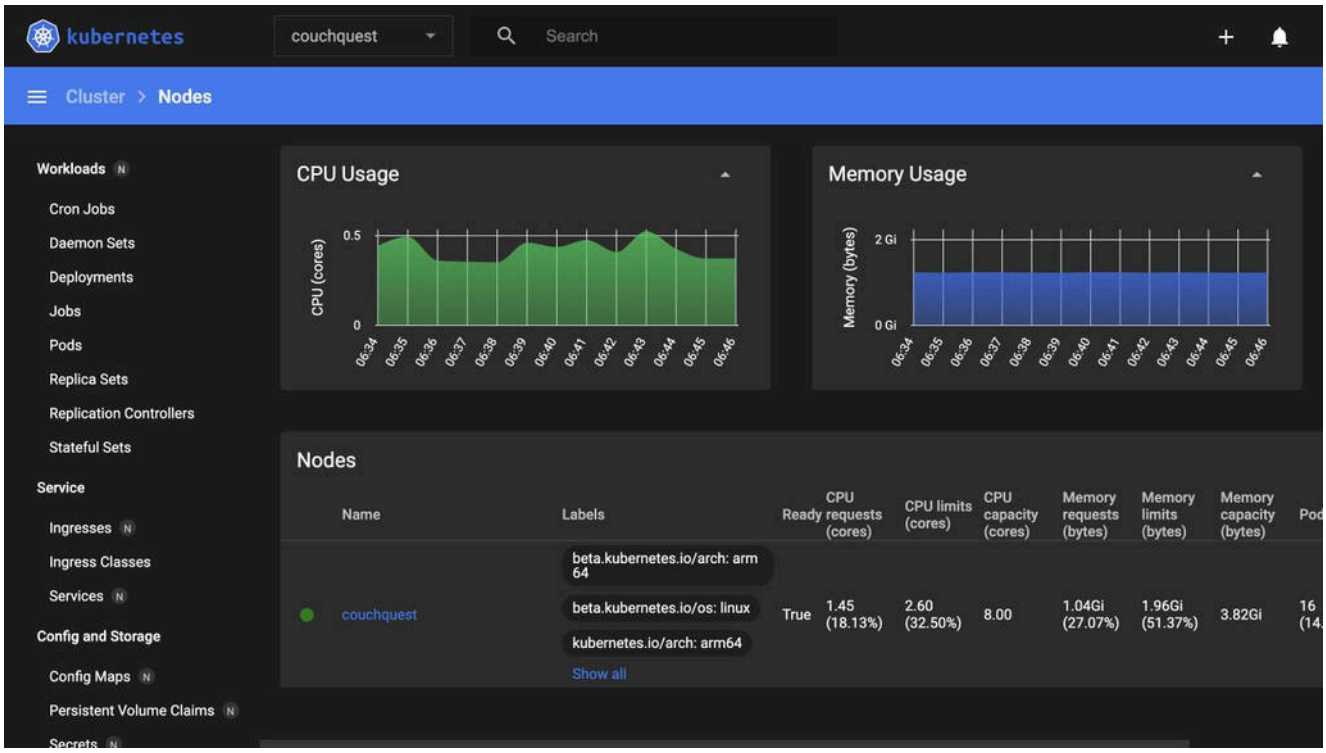

minikube -p couchquest image build \
  -t couchquest-catalog:1.0 catalog-service


minikube -p couchquest image build \
  -t couchquest-session:1.0 session-service
```



The couchquest node is Ready.

The dashboard shows CPU and memory capacity, requests and limits.



The deployment helper creates random local database Secrets if missing, preserves existing credentials, applies workloads and waits for readiness. It always targets couchquest explicitly. Do not commit local credentials.

```
python3 deployment/deploy.py
minikube -p couchquest kubectl -- --context couchquest \
  -n couchquest get deployments,pods,services,pvc
```

The manifests provide one Deployment and internal Service for each API and each database, plus two persistent volume claims. API startup probes allow initialization time. Readiness probes control routing; liveness probes detect an unhealthy process.

Access and inspect

Use separate terminals for the port forwards below. Ports 18001 and 18002 avoid the Docker Compose ports. Set the smoke runner or Postman URLs to these ports. Select couchquest in the dashboard.

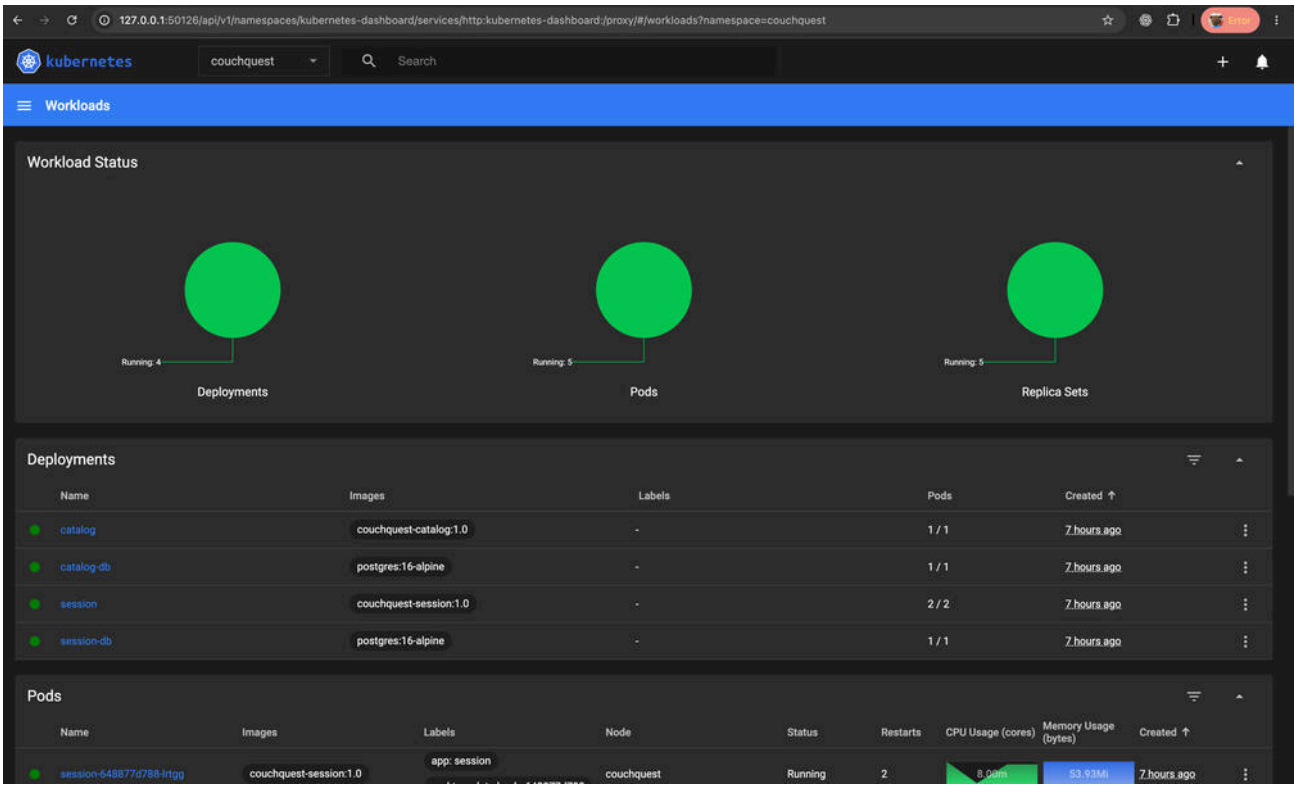
```
minikube -p couchquest kubectl -- --context couchquest \
  -n couchquest port-forward service/catalog 18001:8000
minikube -p couchquest kubectl -- --context couchquest \
  -n couchquest port-forward service/session 18002:8000
minikube -p couchquest dashboard
```

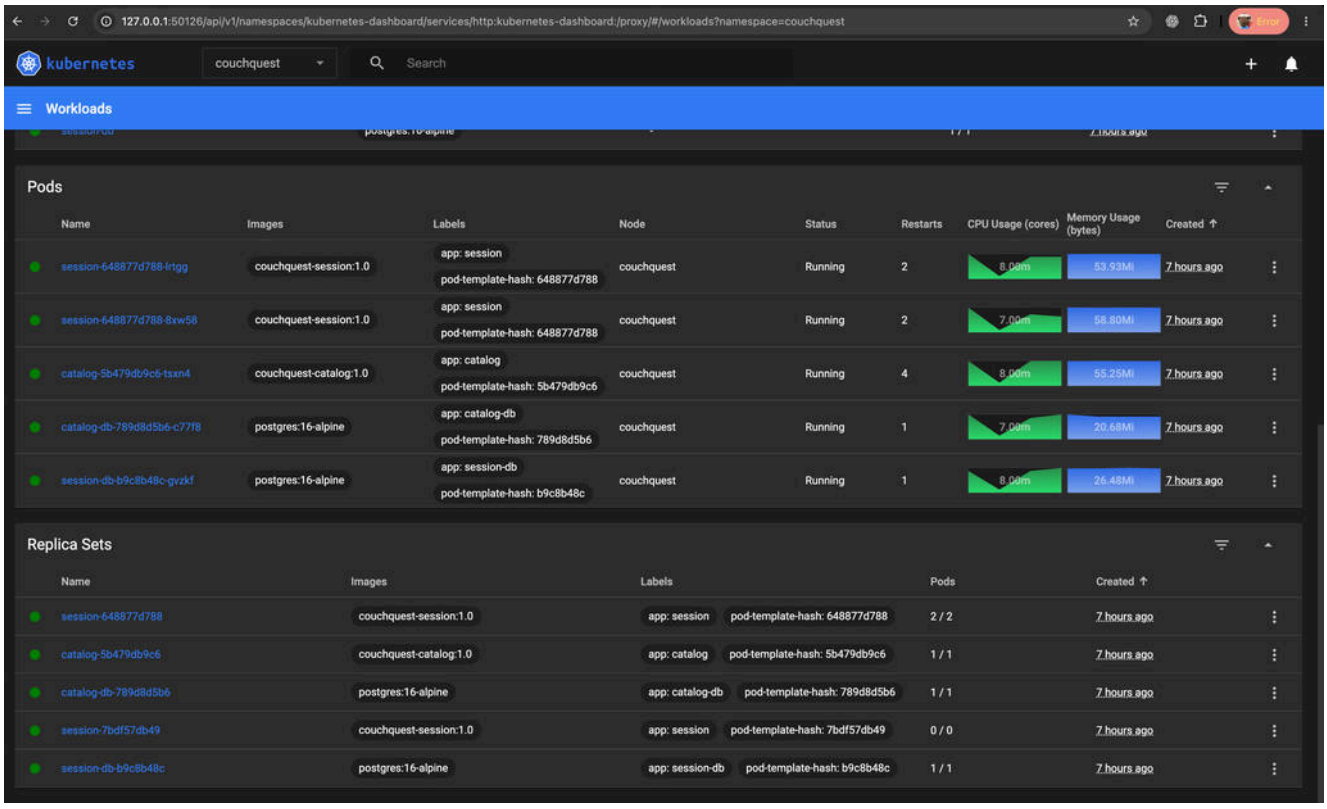
Scale independently

```
minikube -p couchquest kubectl -- --context couchquest \
  -n couchquest scale deployment/session --replicas=2
minikube -p couchquest kubectl -- --context couchquest \
  -n couchquest get pods
minikube -p couchquest addons enable metrics-server
```

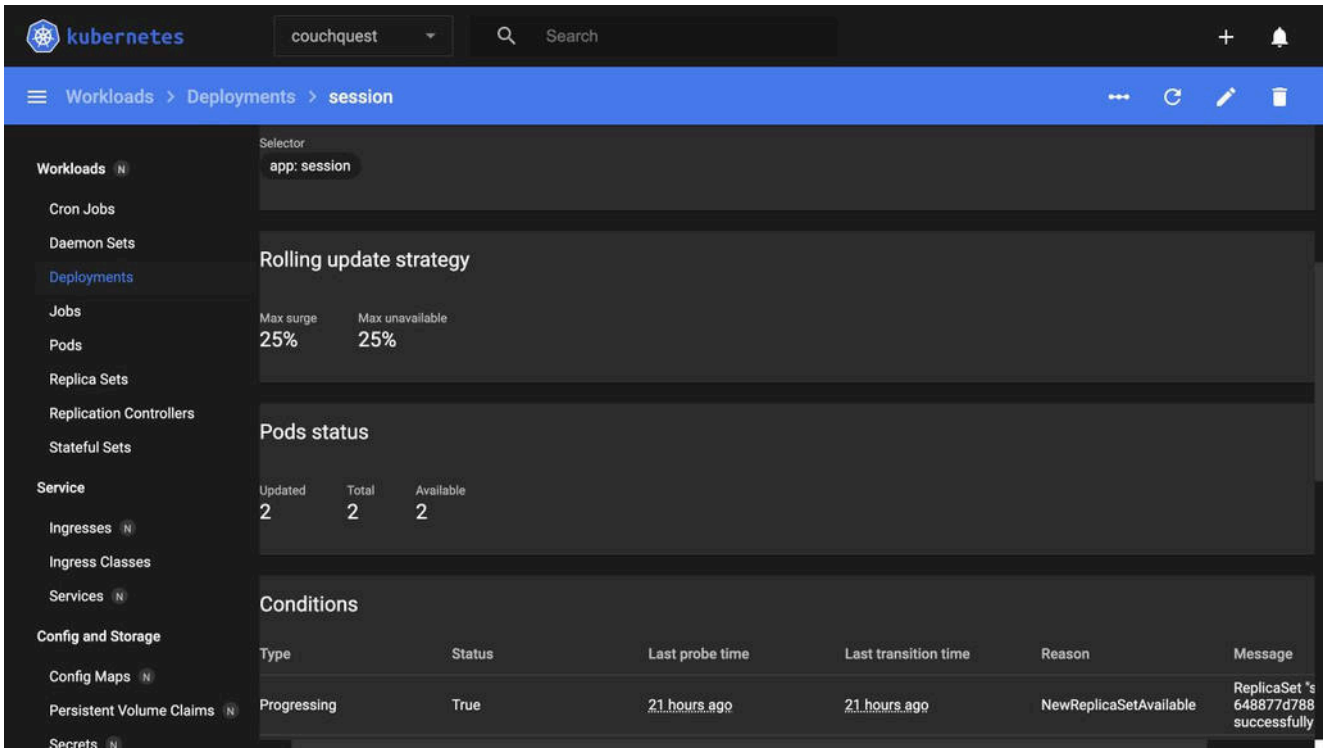
Observe two ready Session pods while the Catalog and database replica counts stay unchanged. HPA needs working metrics and sufficient load; merely applying it does not prove that automatic scale-out occurred.

The dashboard lists independent Catalog, Session and database deployments.

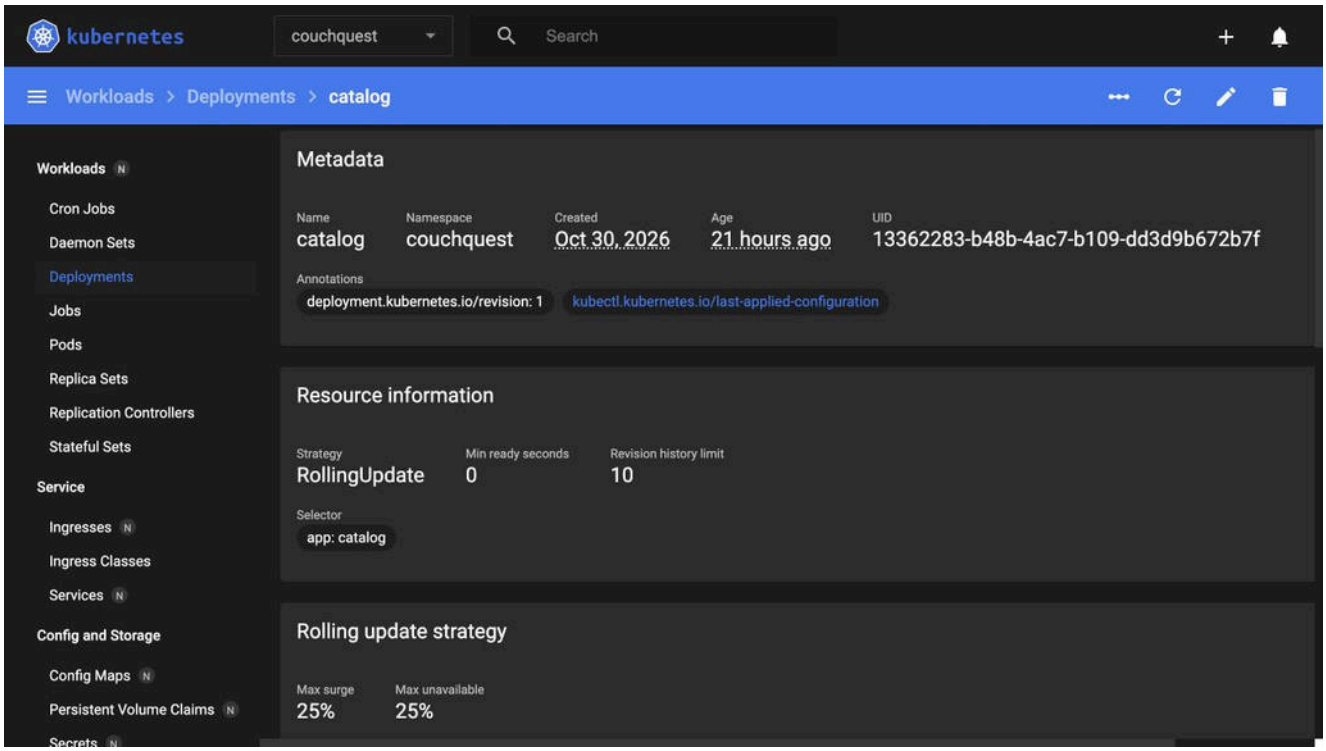




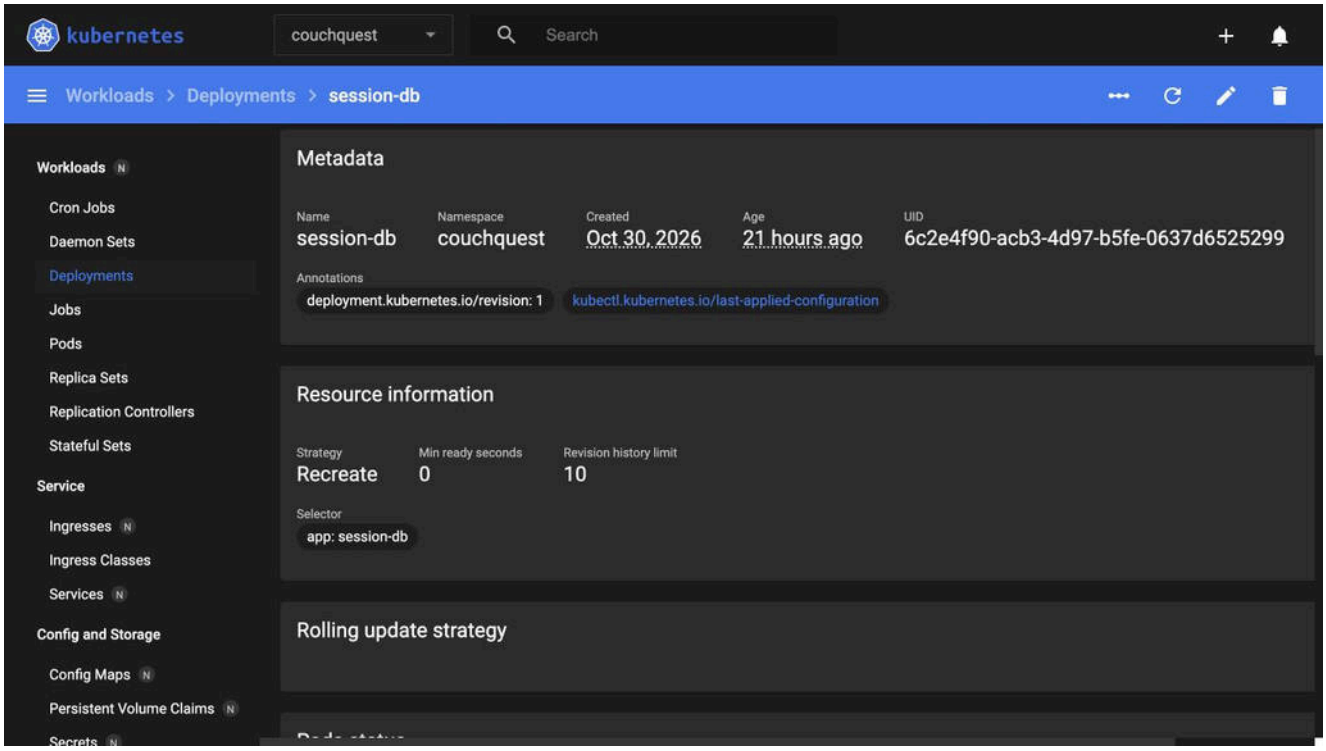
The Session deployment uses RollingUpdate, its own app selector and having two available pods.



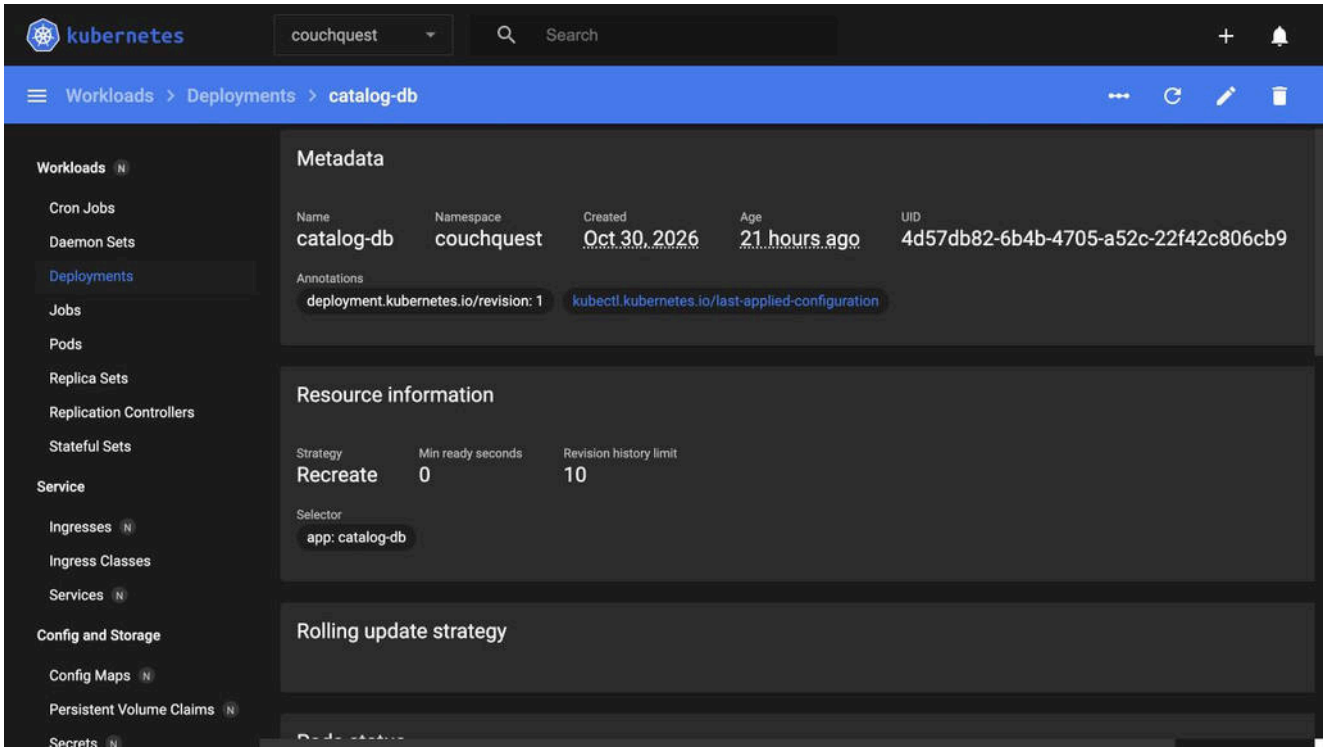
Catalog is an independent deployment using the RollingUpdate strategy and its own app selector.



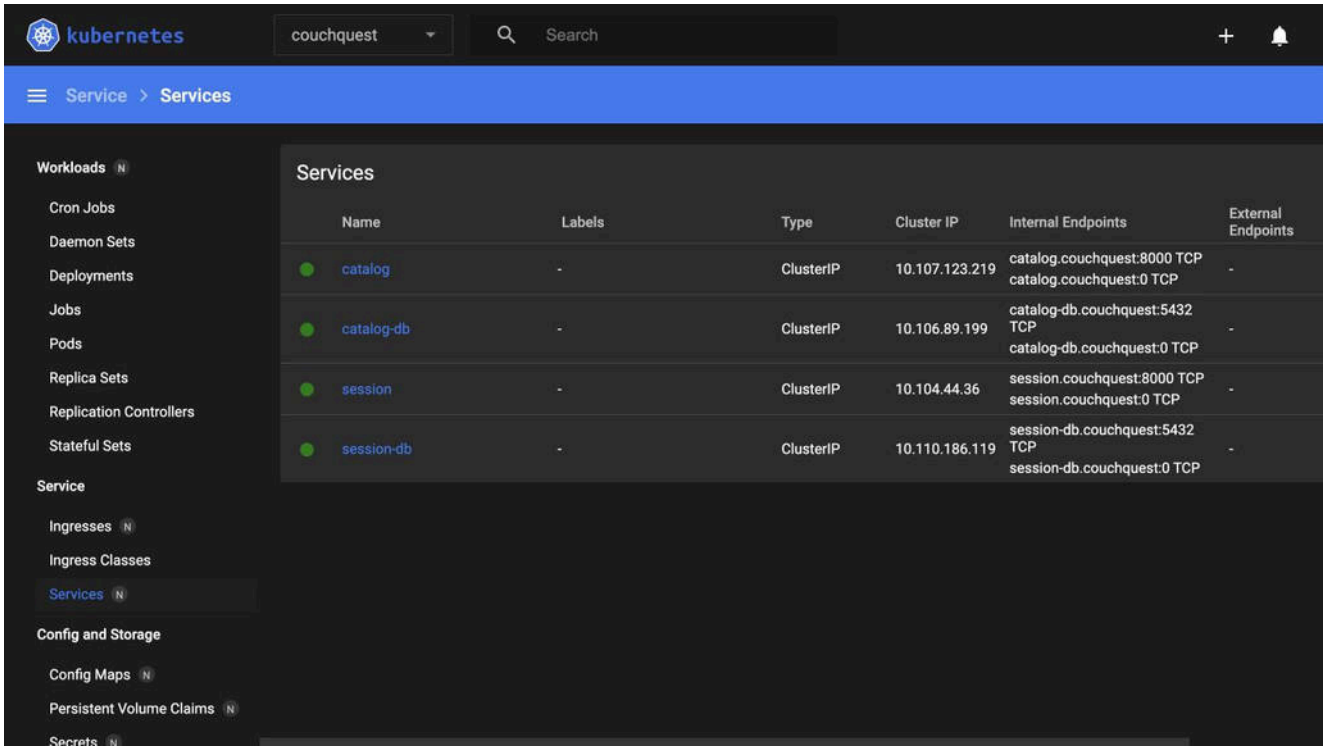
The Session database is deployed separately and uses the Recreate strategy.



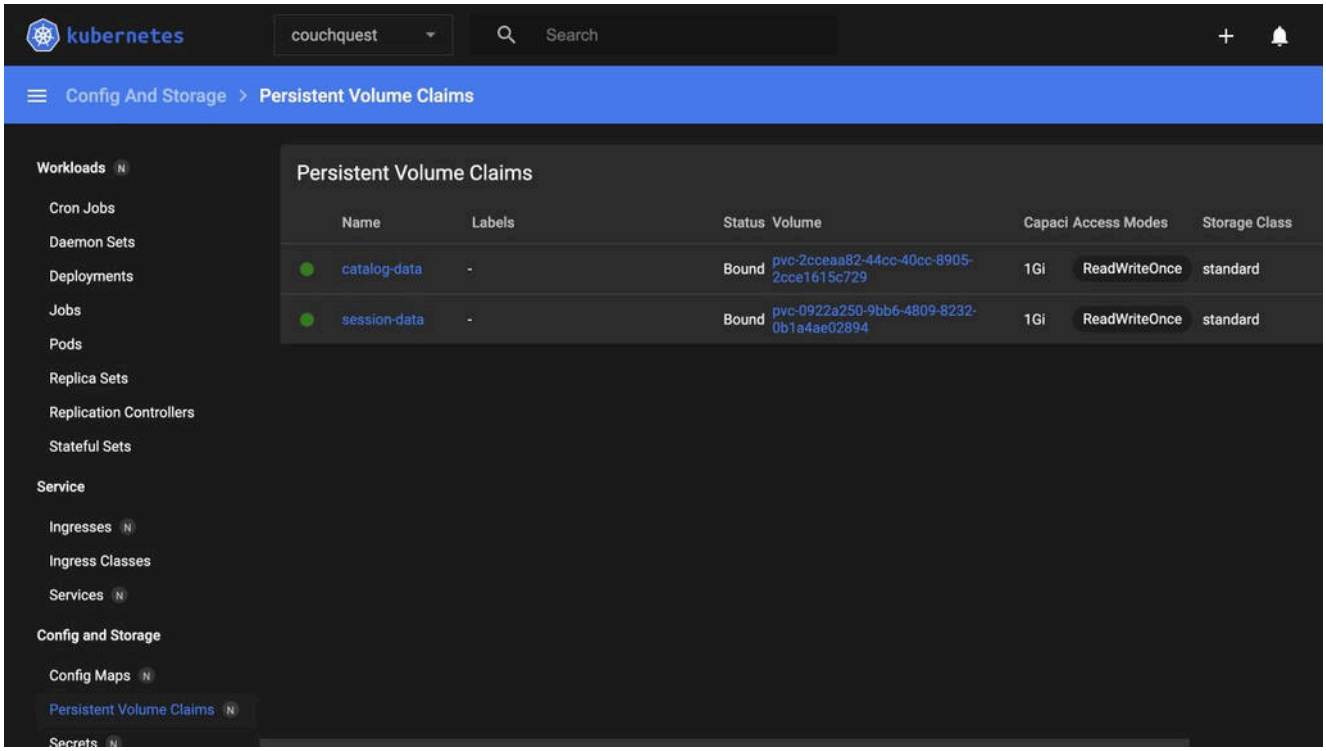
The Catalog database is deployed separately and uses the Recreate strategy.



Separate ClusterIP Services expose the two APIs on port 8000 and their PostgreSQL databases on port 5432



Catalog and Session each have a Bound 1 GiB persistent volume claim with ReadWriteOnce access.



Local URLs

The local URLs below are examples of the deployed environment. They will not open on another computer unless CouchQuest is deployed there and the corresponding containers, Kubernetes port forwards and dashboard proxy are running. Dashboard ports may differ when the environment is restarted.

Work product or resource	Full URL or availability
Catalog application and API documentation — Docker	http://127.0.0.1:8001/docs
Session application and API documentation — Docker	http://127.0.0.1:8002/docs
Catalog application and API documentation — Kubernetes	http://127.0.0.1:18001/docs
Session application and API documentation — Kubernetes	http://127.0.0.1:18002/docs
Catalog readiness — Kubernetes	http://127.0.0.1:18001/health/ready
Session readiness — Kubernetes	http://127.0.0.1:18002/health/ready
Kubernetes dashboard — Deployments	http://127.0.0.1:58018/api/v1/namespaces/kubernetes-dashboard/services/http:kubernetes-dashboard:/proxy/#/deployment?namespace=couchquest
Kubernetes dashboard — Pods	http://127.0.0.1:58018/api/v1/namespaces/kubernetes-dashboard/services/http:kubernetes-dashboard:/proxy/#/pod?namespace=couchquest
Kubernetes dashboard — Services	http://127.0.0.1:58018/api/v1/namespaces/kubernetes-dashboard/services/http:kubernetes-dashboard:/proxy/#/service?namespace=couchquest
Kubernetes dashboard — Persistent volume claims	http://127.0.0.1:58018/api/v1/namespaces/kubernetes-dashboard/services/http:kubernetes-dashboard:/proxy/#/persistentvolumeclaim?namespace=couchquest
Live Session HPA configuration and status — Kubernetes API JSON	http://127.0.0.1:58018/apis/autoscaling/v2/namespaces/couchquest/horizontalpodautoscalers/session
Final assignment report, architecture diagrams and actual Postman/HPA snapshots	Included in the submitted report and supporting files; no separate hosted URL
Demonstration video	Submit as an accompanying video file in the assignment package; no hosted video URL has been established

Postman configuration: The repository collection defaults to Docker ports 8001 and 8002. For Kubernetes testing, set `catalog_url` to `http://127.0.0.1:18001` and `session_url` to `http://127.0.0.1:18002`.

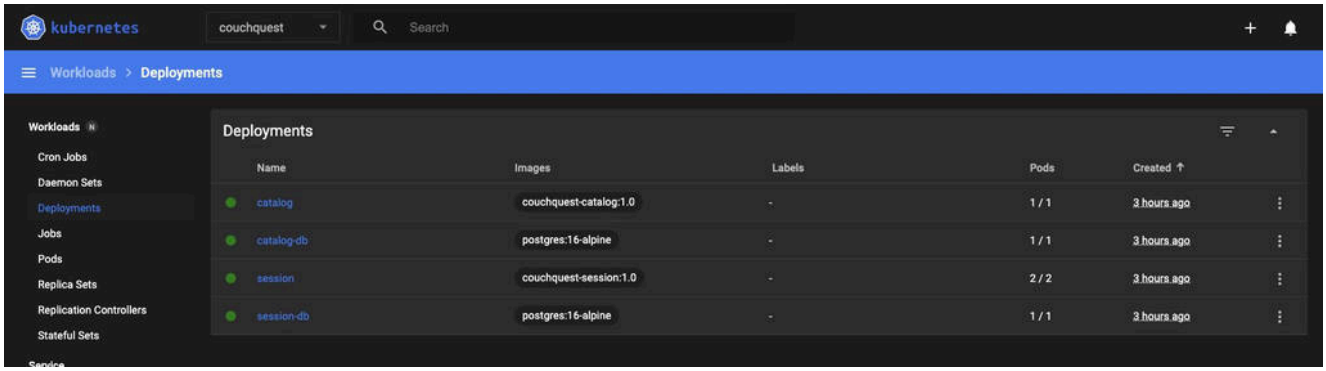
7 Scalability enhancement using HPA

CouchQuest uses a Horizontal Pod Autoscaler (HPA) for the Session deployment in the local Minikube cluster. The autoscaler targets 70% average CPU utilization and manages between one and three Session replicas. Metrics Server supplies the CPU measurements. Each Session container requests 100 millicores, so 70% corresponds to approximately 70 millicores per replica, not 70% of the machine CPU.

Session replicas share the existing Session database; Catalog and both database deployments remain at one replica. HPA now manages the Session replica count instead of the earlier manual two-replica setting.

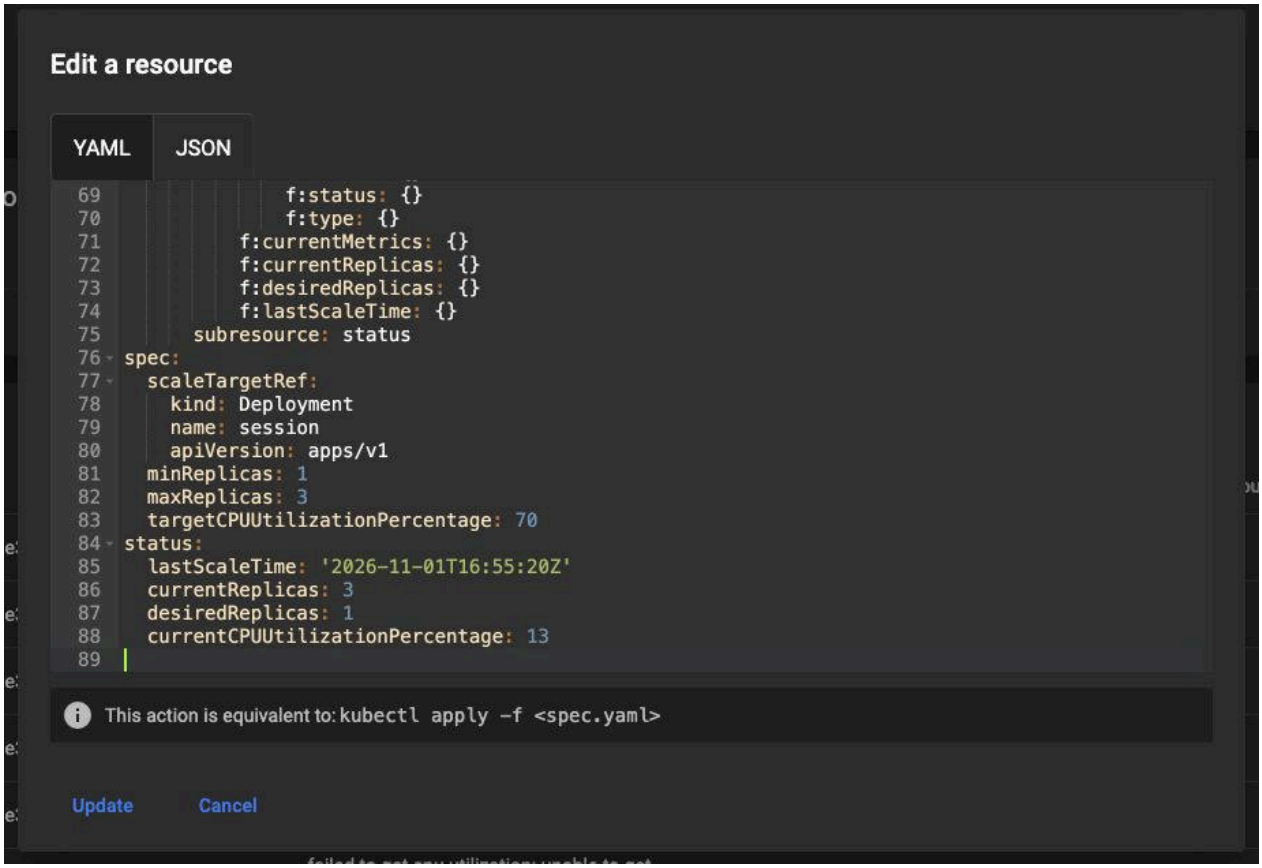
A test with 24 concurrent clients sent read-only requests to an existing Twin Trails session. All 73,905 requests returned HTTP 200. HPA increased Session from one to three replicas when CPU utilization exceeded the target (225% at the scale-up observation; peak sampled utilization 345%). After traffic stopped and the stabilization period elapsed, it reduced Session from three replicas to one. All 11 application smoke checks passed after scaling.

This demonstrates horizontal pod scaling on one local node. It does not demonstrate adding nodes or scaling the database. The default scale-down stabilization window is five minutes, which avoids removing replicas immediately after a brief drop in demand.



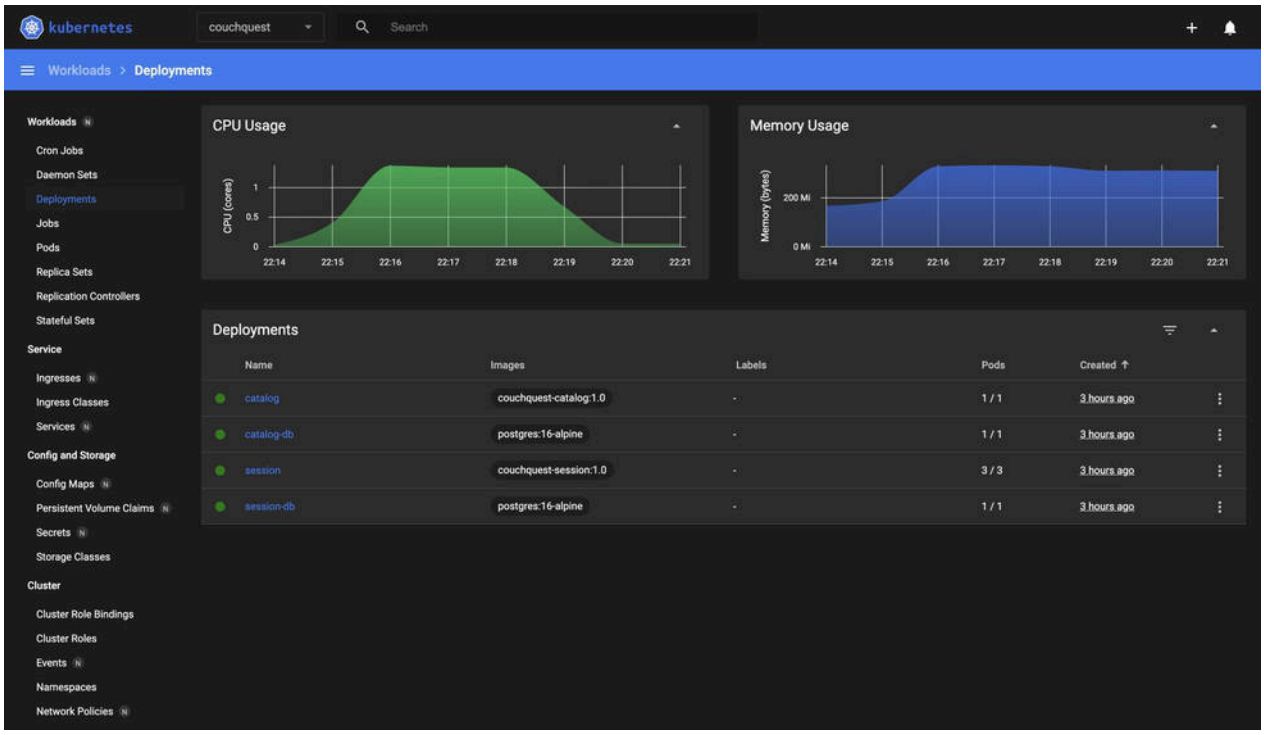
Initial deployment: two manually configured Session replicas; Catalog and both databases each have one replica.

HPA configuration



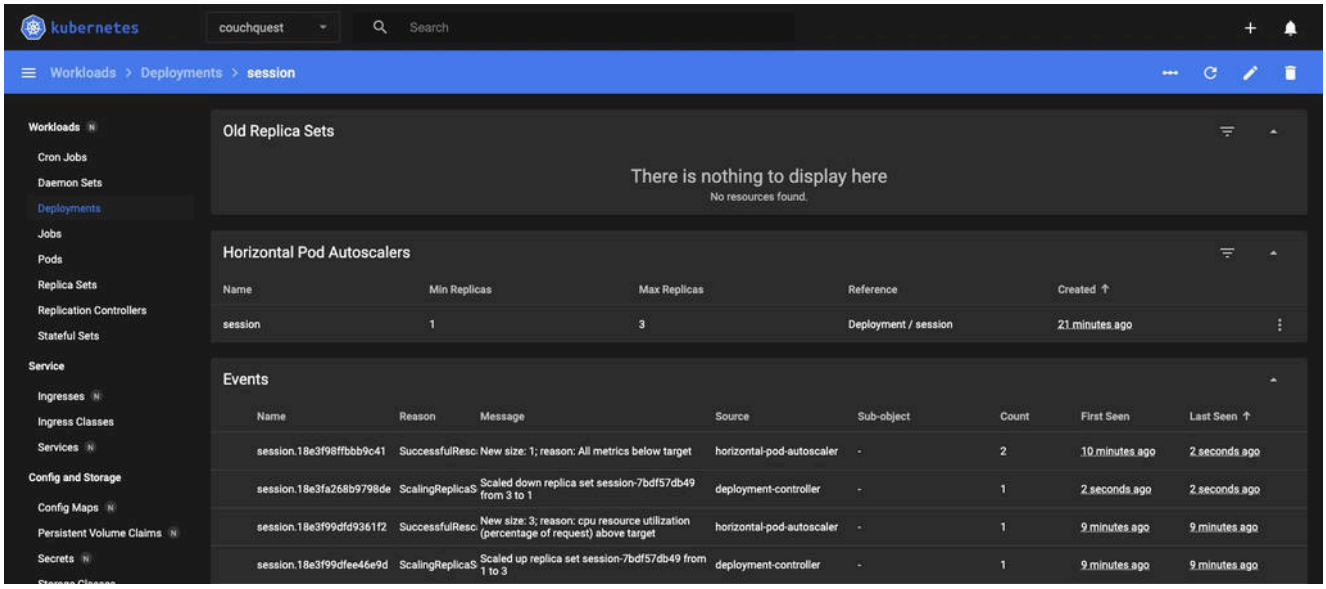
Live HPA specification: Session target, minimum 1, maximum 3 and CPU target 70%. Captured during scale-down: current 3, desired 1, CPU 13%. The configuration viewer was closed without edits.

Scale up under application traffic



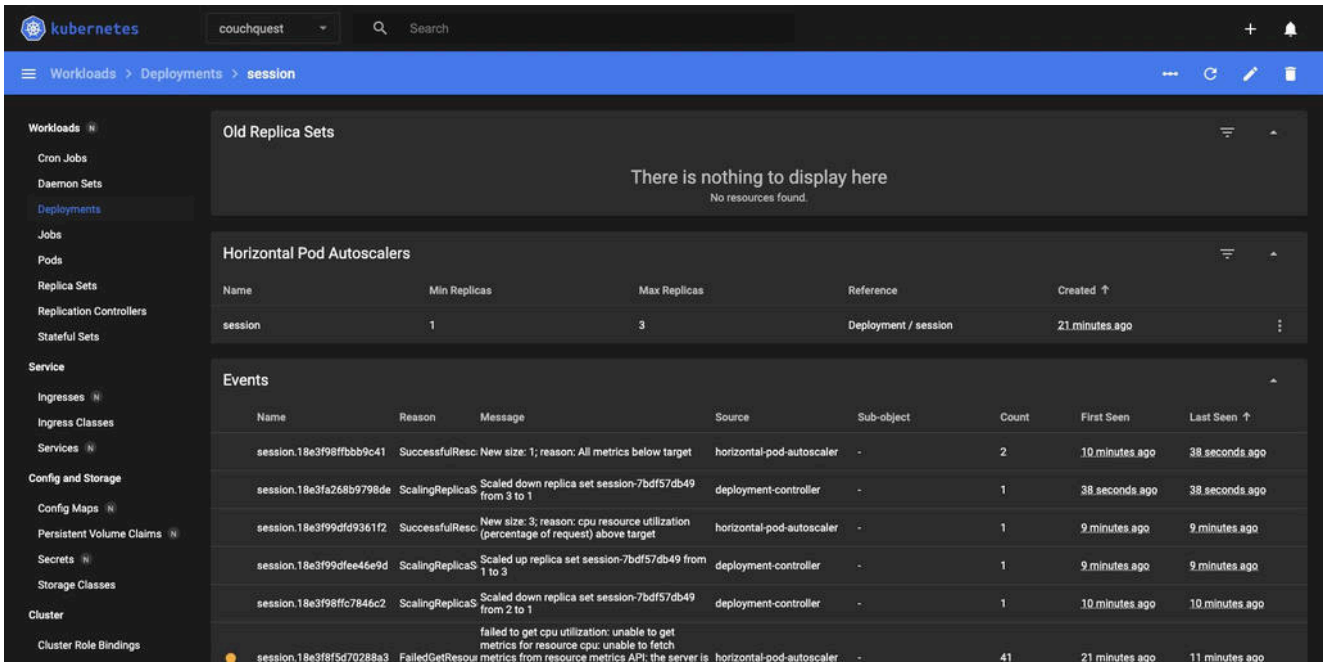
Session reached three healthy replicas. The CPU history shows the load peak followed by falling usage; the capture was taken during the stabilization period after traffic stopped.

Automatic scale up recorded by Kubernetes



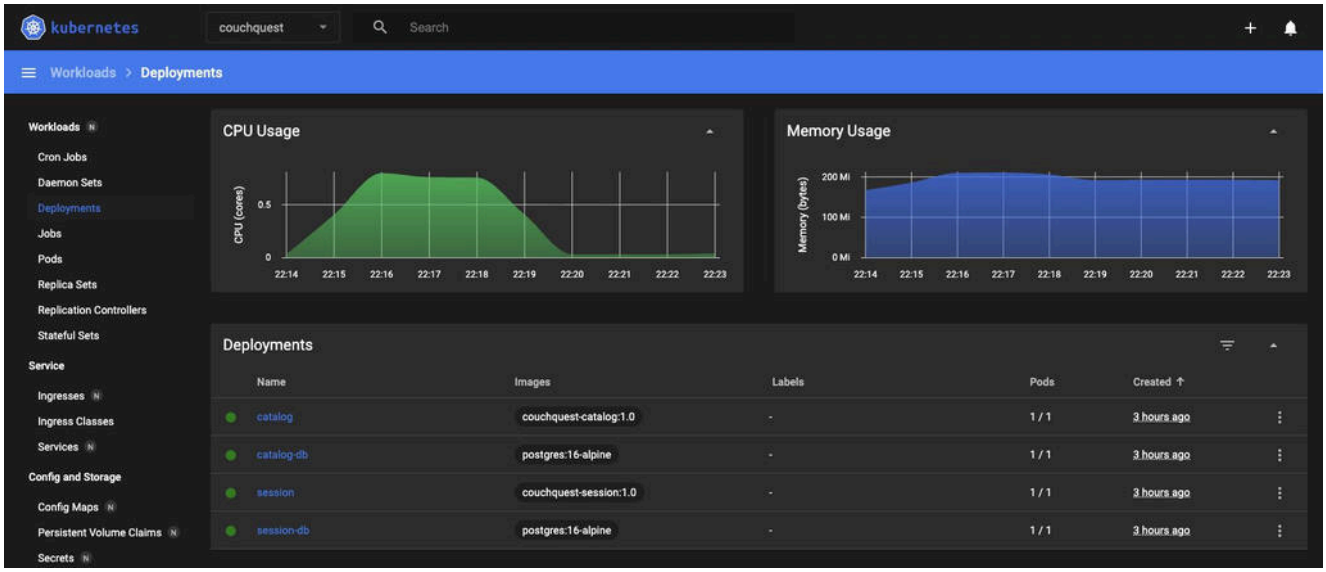
SuccessfulRescale records a new size of 3 because CPU was above target. The deployment controller confirms the increase from one to three replicas.

Automatic scale down after traffic stopped



SuccessfulRescale and ScalingReplicaSet record the reduction from three replicas to one. Older metrics warnings at the bottom relate to Metrics Server startup and were resolved before testing.

Final healthy deployment

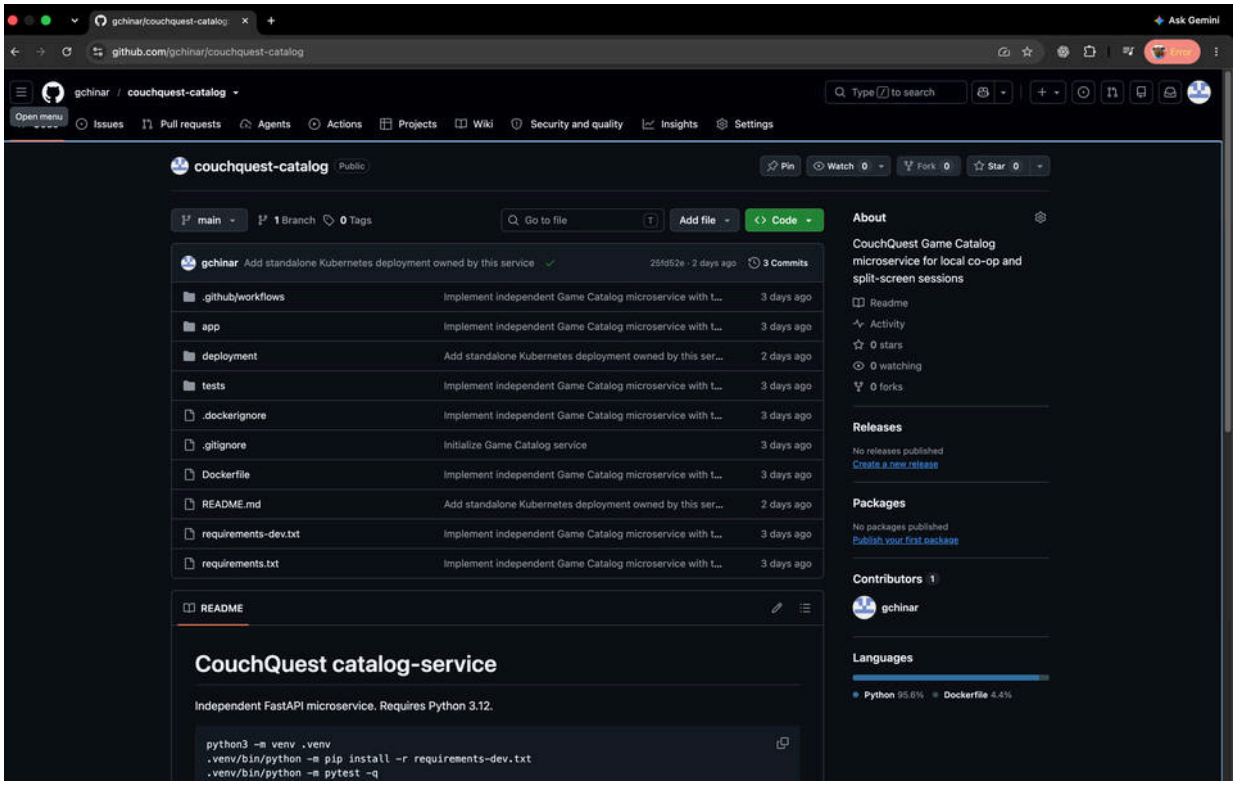


Final state: one healthy Session replica, with Catalog and both databases also healthy. HPA remains enabled and continues managing between one and three Session replicas.

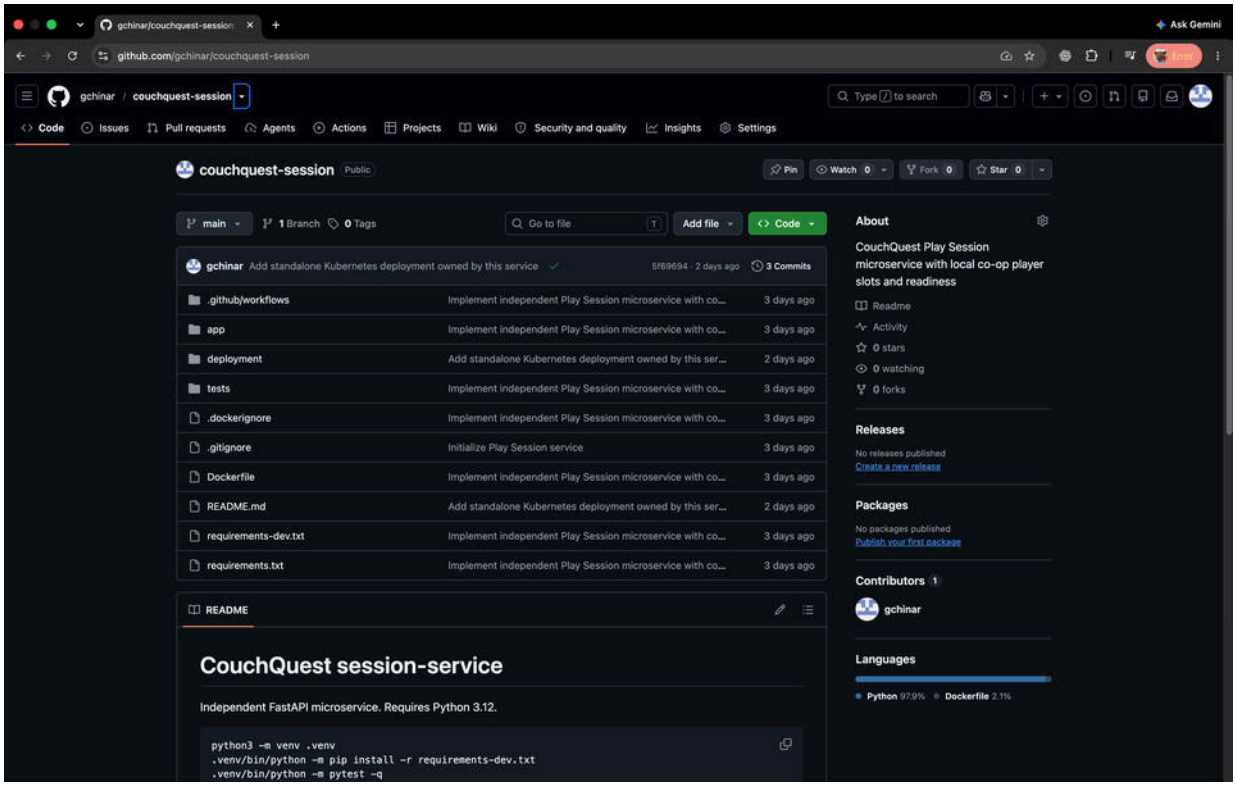
8 Repositories

Catalog and Session have separate repositories, each with its own dependencies, tests, Dockerfile, deployment files and CI workflow. The coordinating repository contains documentation, shared deployment configuration and pinned Git submodules.

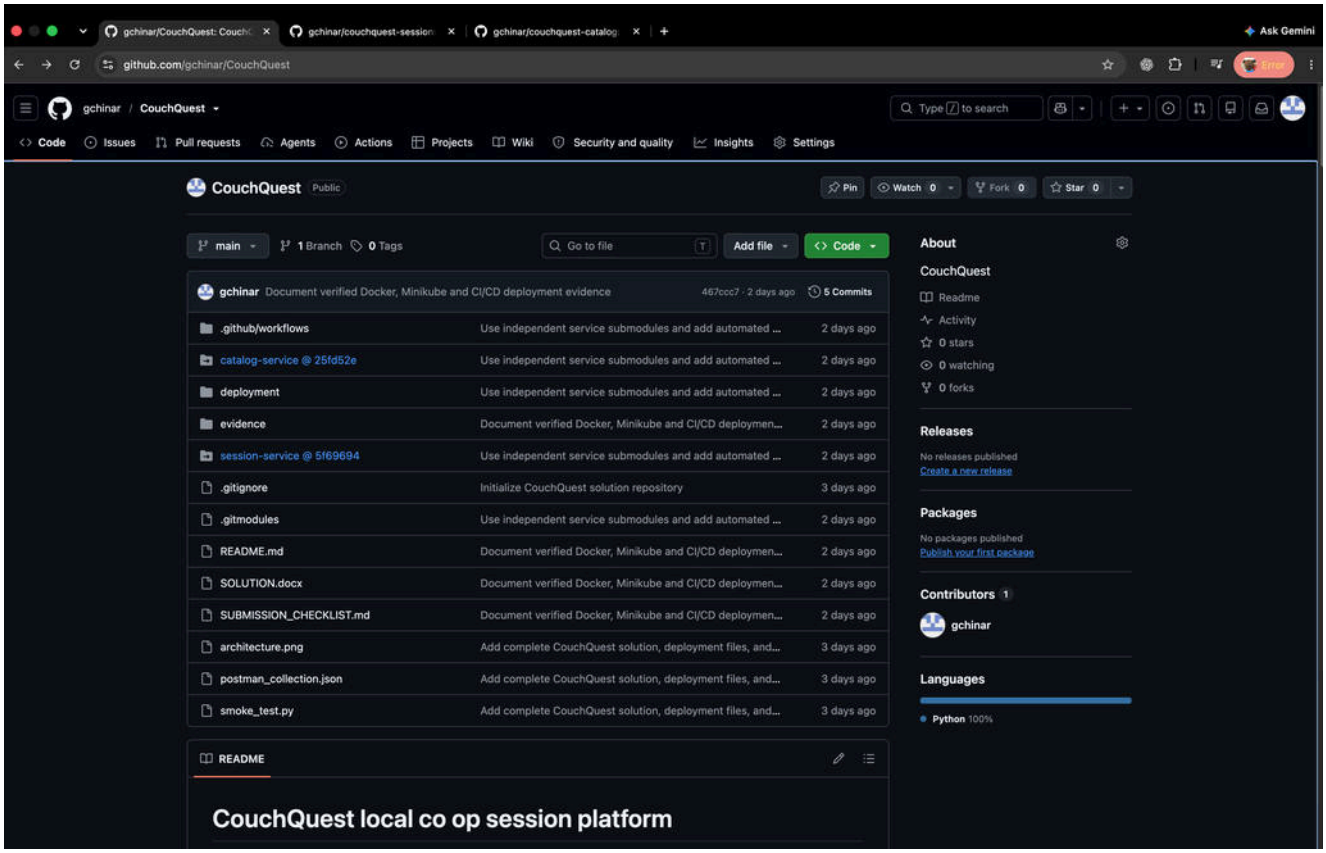
Catalog repository https://github.com/gchinar/couchquest-catalog



Session repository https://github.com/gchinar/couchquest-session



Complete solution https://github.com/gchinar/CouchQuest



9 CI and CD execution

Options considered

GitHub-hosted Minikube was selected because deployment can be repeated without long-lived cloud credentials. A self-hosted runner could deploy to a persistent local cluster but needs maintenance and isolation from untrusted changes. A registry and GitOps controller could promote immutable images to a persistent production cluster; that extension is not configured.

Independent releases

Catalog and Session each retain their own CI and deployment manifests. The coordinating repository pins tested commits as submodules. Promoting a new service commit updates its pin; changes to deployment inputs or pins trigger the integration pipeline on main. Manual workflow execution is also available. This provides independent development and testing with an explicitly versioned integration deployment.

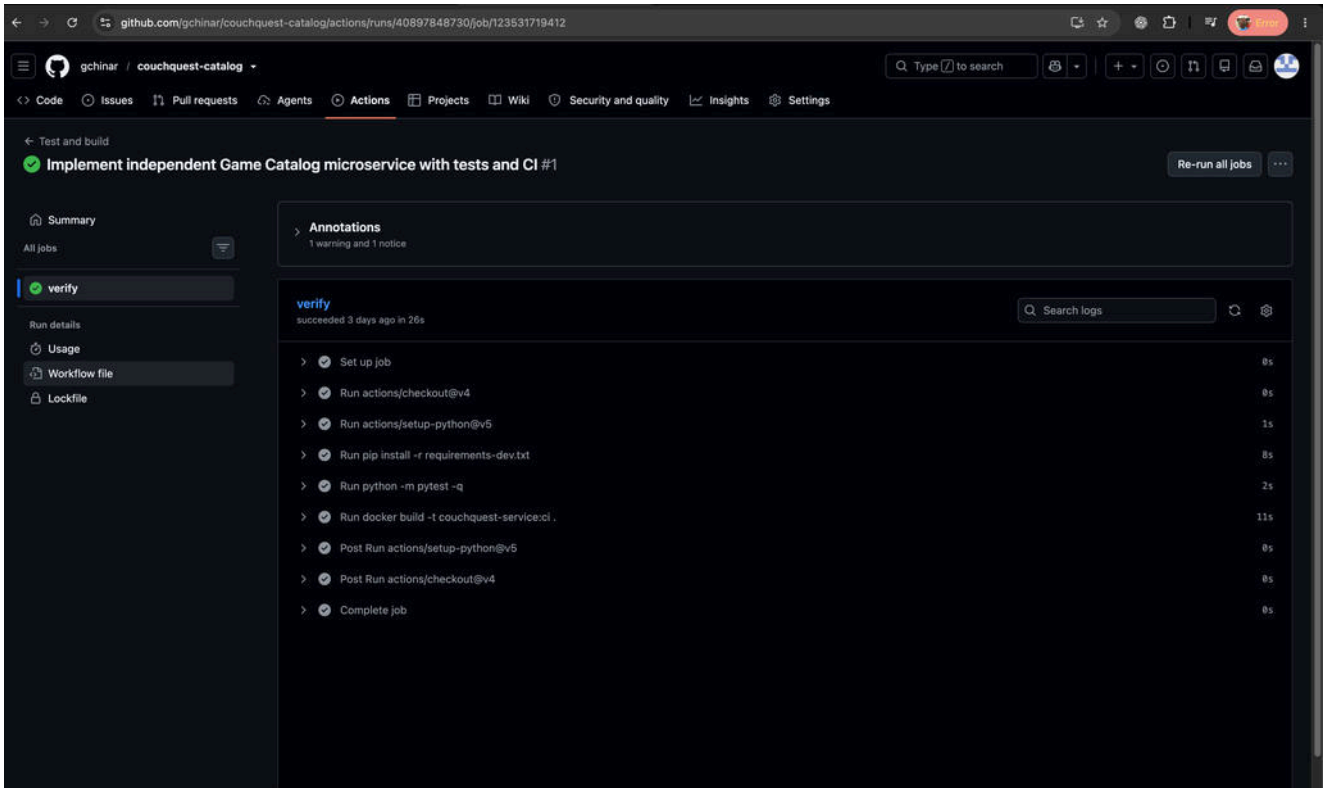
The implemented GitHub Actions pipeline tests the independently maintained services, builds their container images, creates a temporary Minikube cluster, deploys the application and databases, waits for rollouts, runs the complete HTTP scenario and uploads diagnostics.

The test-build-deploy job completed successfully: independent tests, image builds, Kubernetes deployment, HTTP checks and evidence upload all passed. Machine-readable run details are in evidence/deployment/ci-run.json. The workflow is .github/workflows/ci.yml.

https://github.com/gchinar/couchquest-session/blob/main/.github/workflows/ci.yml

https://github.com/gchinar/couchquest-catalog/blob/main/.github/workflows/ci.yml

Catalog and Session have separate repositories, each with its own dependencies, tests, Dockerfile, deployment files and CI workflow. The coordinating repository contains documentation, shared deployment configuration and pinned Git submodules.



https://github.com/gchinar/couchquest-catalog/actions/runs/40897848730/job/123531719412

https://github.com/gchinar/couchquest-session/actions/runs/40897920288/job/123531975579

